# DBMS


ORACLE
firebird
PostgreSQL
IBM DB2
Microsoft SQL Server 2008
SQLite
MySQL
123
TERADATA
CSV

# 강의 목차

DBMS



I. 데이터베이스와 SQL
   1. 데이터베이스란 무엇인가
   2. 데이터베이스 구성
   3. SQL개요
   4. 테이블작성
   5. 테이블 삭제와 변경

II. 검색의 기본
   1. SELECT문의 기본
   2. 산술연산자와 비교연산자
   3. 논리 연산자

# 강의 목차

DBMS



III. 집약과 정렬

1. 집약함수와 검색
2. 그룹핑
3. 조건을 지정한 집약
4. 검색결과의 재정렬

IV. 데이터 갱신

1. 데이터 등록
2. 데이터 삭제
3. 데이터 갱신
4. 트랜잭션

# I. 데이터베이스와 SQL – 1. 데이터베이스란 무엇인가

- Database(DB)란 ?
  - 수많은 데이터를 컴퓨터를 사용하여 효율적으로 접근할 수 있도록 가공한것.
  - 데이터를 데이터베이스에 저장하여 두면
  - 언제든지 간단하게 자신이 원하는 데이터를 꺼낼 수 있다.
- DBMS (Database Management System )데이터베이스 관리 시스템 이란?
  - 데이터베이스 DB를 관리하는 컴퓨터 시스템을 DBMS라 한다.

✓DBMS 가 필요한 이유

DBMS는 대량의 데이터를 다수의 사람이 안전하고 간단히 다룰 수 있게한다.

고도의 프로그래밍 기술이 없어도 사용할 수 있다.

텍스트 파일이나 표 계산프로그램등으로 관리했을때...

- 다수의 사람이 데이터를 공유하기 어렵다.
- 대량의 데이터를 공유하기 어렵다.
- 읽기/쓰기를 자동화하려면 프로그래밍 기술이 필요함.
- 만일의 사고에 대응하기 어렵다.

## DBMS 의 종류

- **계층형 데이터베이스 (Hierarchical Database)**

  가장 오래된 데이터베이스 형태로, 데이터를 계층구조(트리구조)로 표현. 거의사용안함

- **객체지향 데이터베이스(Object Oriented Database : OODB)**

  데이터와 데이터처리를 모아서 객체 단위로 관리한다.

  객체지향 데이터베이스는 이 객체를 저장하기 위한 데이터 베이스임

- **XML 데이터베이스(XML Database : XMLDB)**

  최근 인터넷에서 데이터를 교환하기 위해 XML형식을 자주 사용함.

  이 XML형식 데이터를 대량으로 빠르게 처리하기위해 고안된 XML 데이터베이스

- **키-밸류형 데이터스토어(Key-Value Store:KVS)**

  검색에 사용하는 키와 값을 조합하여 단순한 형태로 데이터를 저장하는 DB.

  구글등 대량의 데이터를 초고속으로 검색해야 하는 웹 서비스 등에 사용되면서 주목됨

# DBMS 의 종류

●관계형 데이터베이스(Relational Database: RDB)

- 현재 가장 폭넓게 이용되고 있음. (1969)
- 엑셀시트처럼 열과 행으로 이루어진 2차원 표형식으로 데이터를 관리 .
- SQL(Structured Query Language:구조화질의구문)이라는 전용 언어를 사용하여 데이터 처리.
- 관계형 데이터 베이스의 데이터 구조예

| 상품ID | 상품명 | 상품분류 | 판매단가 | 등록일 |
|---|---|---|---|---|
| 0001 | 티셔츠 | 의류 | 1000 | 2015-04-05 |
| 0002 | 목걸이 | 악세서리 | 5000 | 2015-04-09 |
| 0003 | 벨트 | 악세서리 | 4000 | 2016-04-05 |
| 0004 | 구두 | 슈즈 | 12000 | 2017-04-09 |

# DBMS 의 종류

●관계형 데이터베이스(Relational Database: RDB)

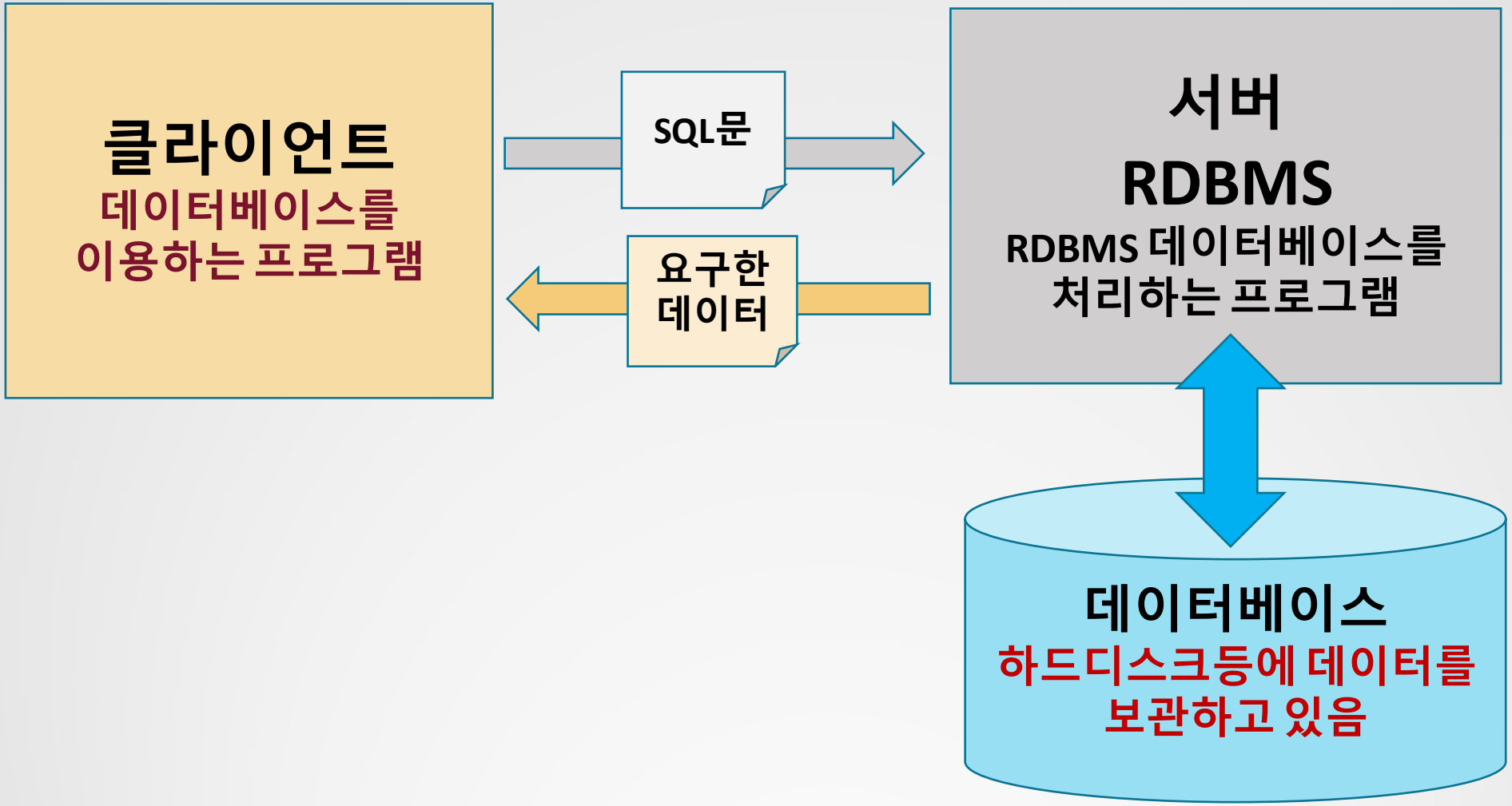
- 대표적인 RDBMS 는 다섯가지가 있다.

| | |
|---|---|
| Oracle | Oracle |
| SQL Server | 마이크로소프트 RDBMS |
| DB2 | IBM 의 RDBMS |
| PostgreSQL | 오픈소스 RDBMS |
| MySQL | 오픈소스 RDBMS |

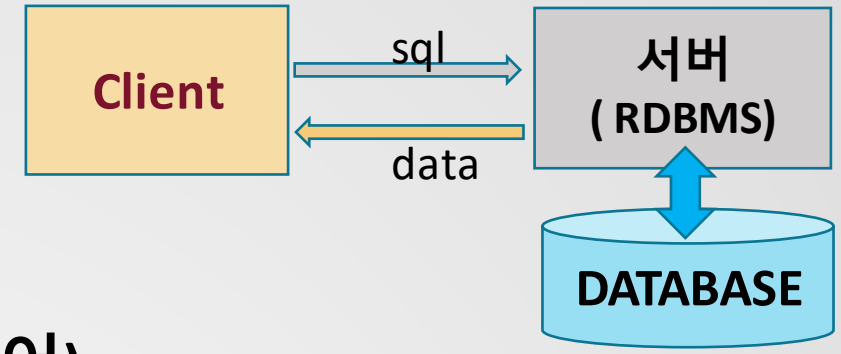
- 우리가 배우는 DBMS도 SQL을 사용한 RDBMS를 조작하는 방법을 배울 것이다.
- RDBMS중에는 XML,객체지향등의 데이터베이스 기능을 채용하고 있는 것도 있으나 이러한 확장기능을 사용하기 위한 SQL을 다루지는 않을것임.

# I. 데이터베이스와 SQL – 2. 데이터 베이스 구성

## RDBMS 시스템 구성 ( 클라이언트-서버형) – 가장 일반적인 시스템구성

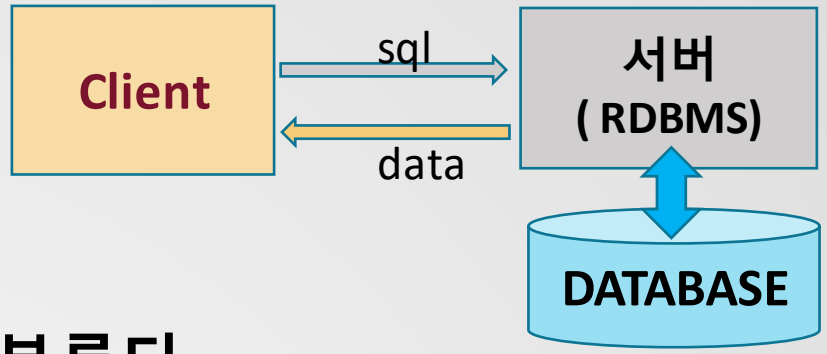


## 서버란? ( 심부름꾼 )




- 다른 프로그램으로부터 요청을 받아
  해당 요청에 따라 처리를 하는 프로그램(소프트웨어)
  또는 그 프로그램이 설치된 장비(컴퓨터)fmf dmlalgksek.
  컴퓨터에서 계속 동작하면서 요청이 올때까지 대기한다.
  RDBMS도 서버의 일종으로, 하드디스크등에 저장된 데이터베이스로부터
  데이터를 꺼내서 반환하거나, 지시된 내용으로 데이터를 변경한다.

## 클라이언트 ( 의뢰인 )

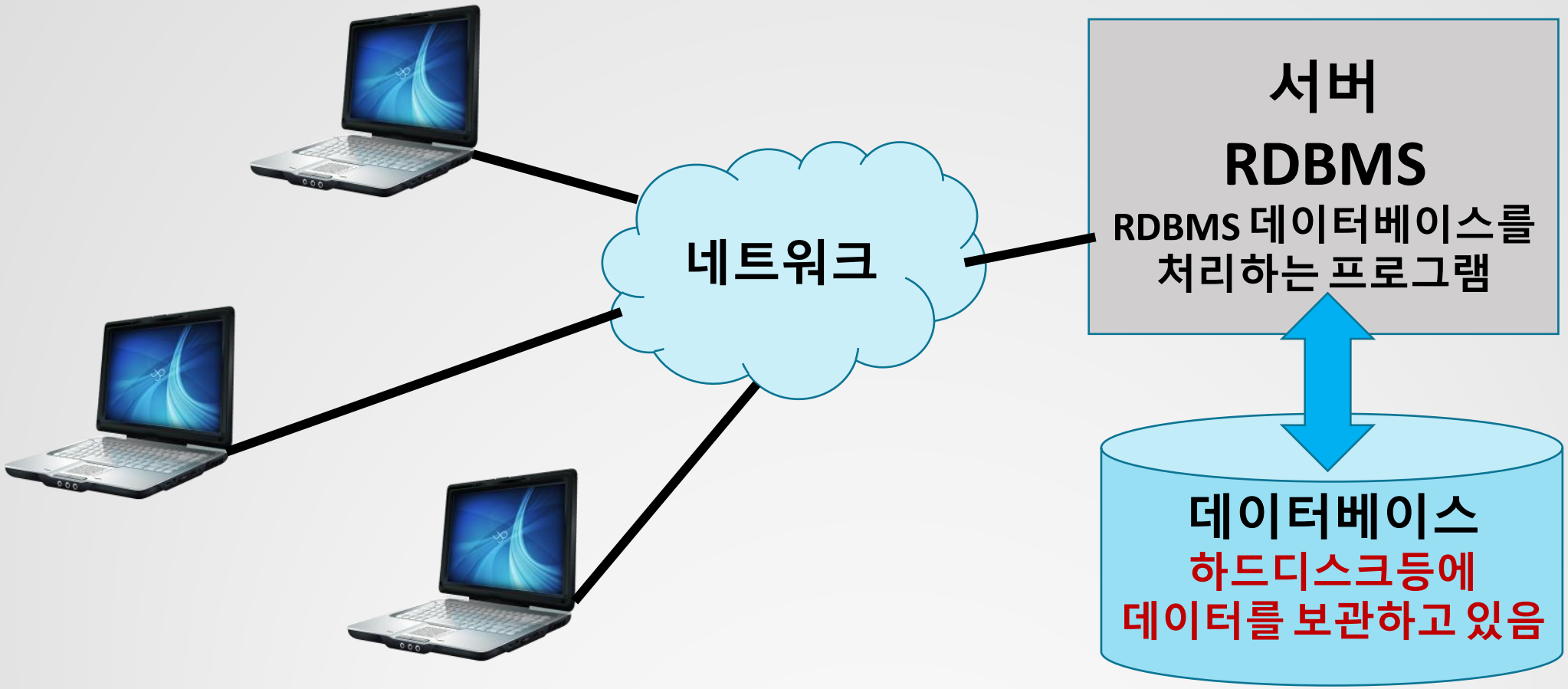



- 서버에 요청을 던지는 프로그램(소프트웨어)

  또는 그 프로그램이 설치된 장비를 클라이언트라 부른다.

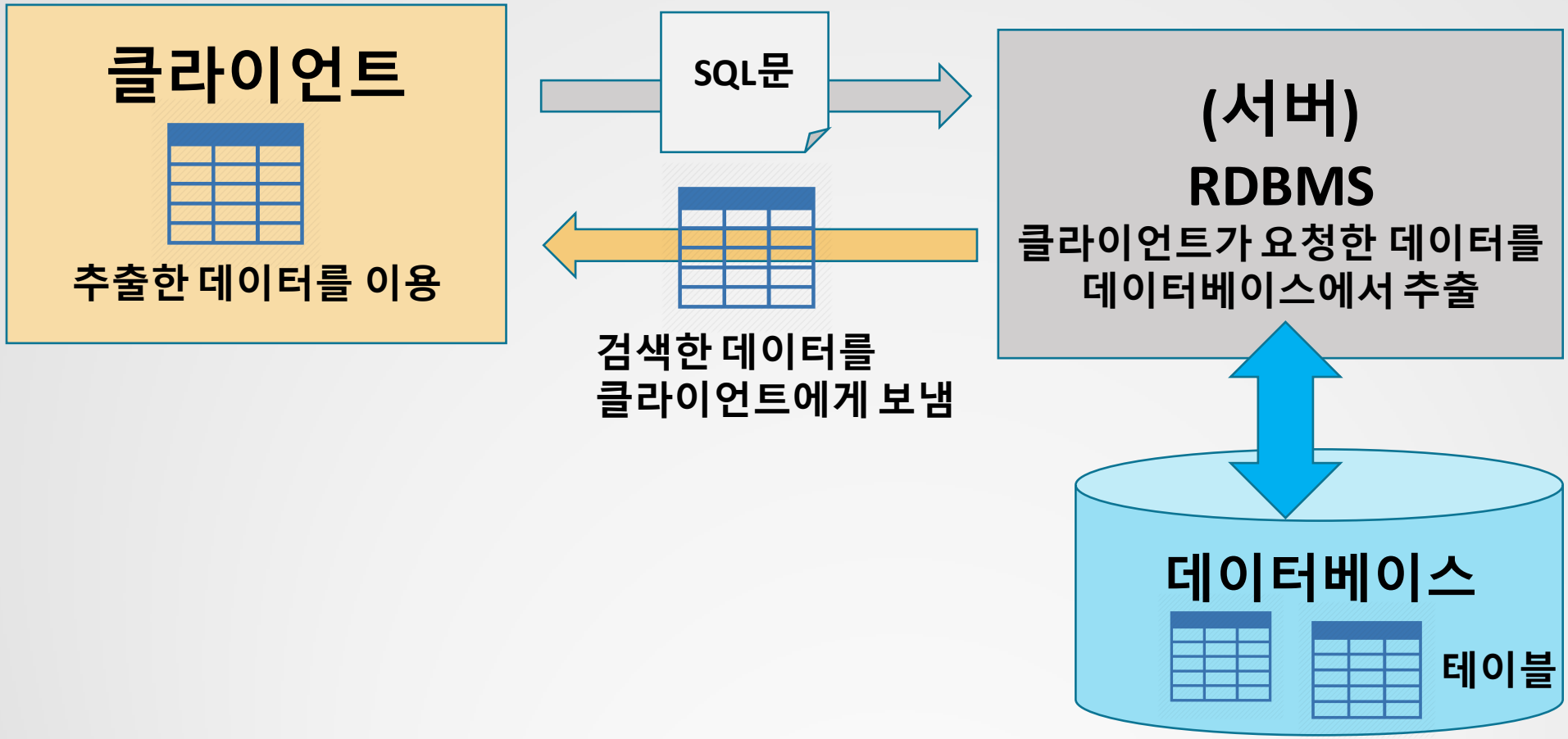
  클라이언트는 어떤 데이터를 어떻게 처리하고 싶은지 SQL 구문으로 작성하여 서버인 RDBMS로 전송한다.

# 네트워크 경유로 하나의 데이터베이스에 복수의 클라이언트가 접속

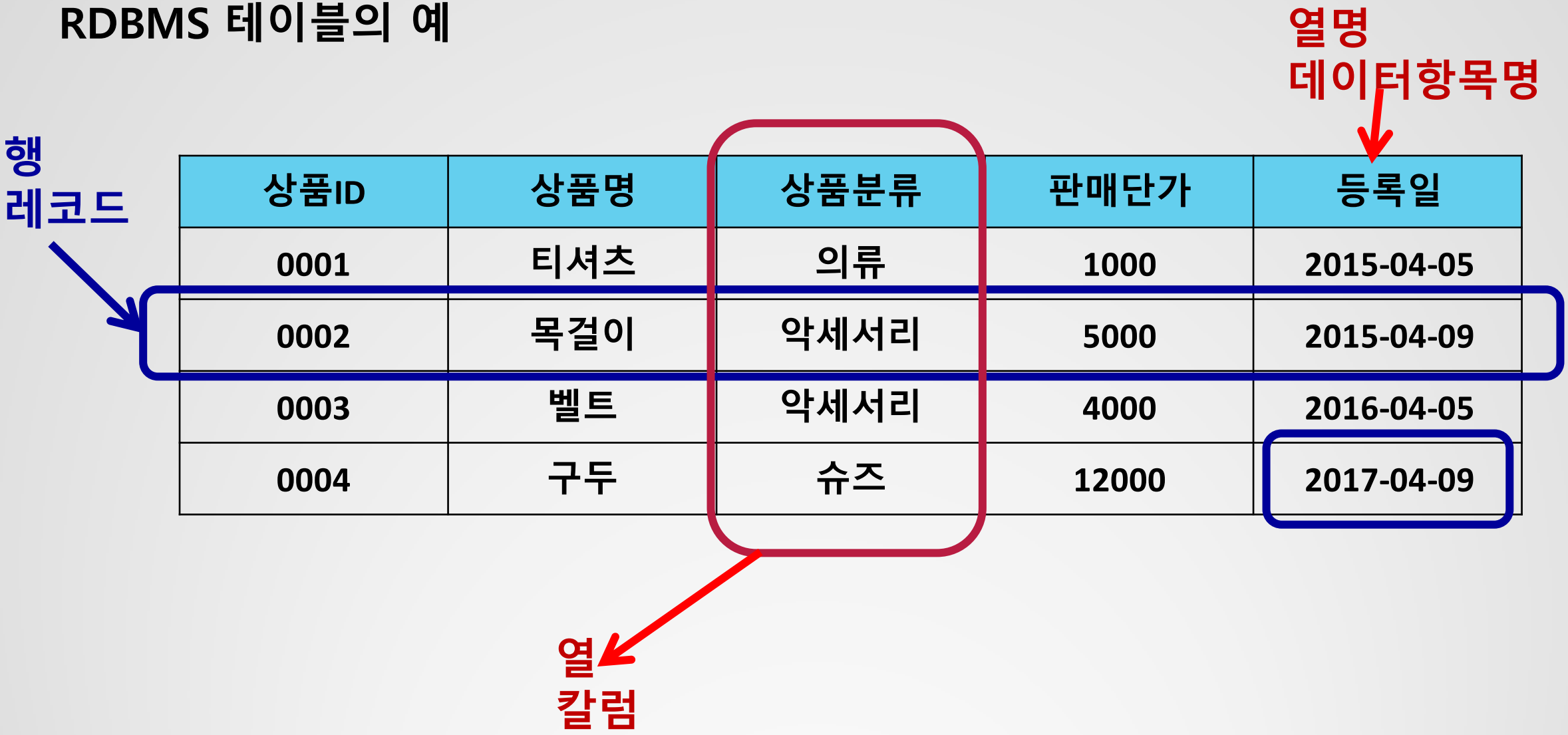


# 관계형 데이터 베이스와 테이블

- 데이터를 관리하는 2차원표를 테이블이라한다.
- 테이블은 RDBMS가 관리하는 데이터베이스 안에 저장된다.



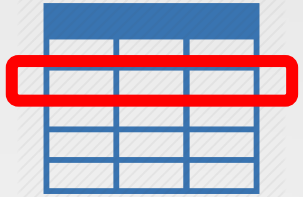
# RDBMS 테이블의 예



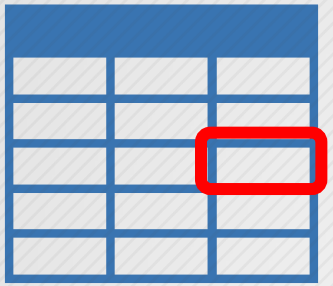

| 상품ID | 상품명 | 상품분류 | 판매단가 | 등록일 |
|---|---|---|---|---|
| 0001 | 티셔츠 | 의류 | 1000 | 2015-04-05 |
| 0002 | 목걸이 | 악세서리 | 5000 | 2015-04-09 |
| 0003 | 벨트 | 악세서리 | 4000 | 2016-04-05 |
| 0004 | 구두 | 슈즈 | 12000 | 2017-04-09 |

## 관계형 데이터베이스 특징

- 반드시 행단위로 데이터를 읽고 쓴다.



- 하나의 셀안에는 하나의 데이터만넣을 수 있다.



- RDBMS는 중요한 데이터를 아무나 열람수정못하도록 등록한 사용자만 데이터베이스에 접근할 수 있다. 이 사용자는 OS에 등록된 사용자가 아니며 별도로 RDBMS 에서만 인식된다. RDBMS는 다수의 사용자를 등록할 수 있다.

# I. 데이터베이스와 SQL – 3. SQL 개요

**SQL ( Structured Query Language)**

- SQL 은 관계형데이터베이스를 제어하기 위해 개발된 언어이다.
- SQL은 표준규격이 존재하지만, 실제로는 RDBMS별로 차이가 있다.
- 표준 SQL 은 IOS에서 정한 표준 규격을 따르는 SQL이다. 표준 SQL을 익히면 다양한 RDBMS에서 SQL문을 작성할 수 있다.
- SQL에서는 처리하고 싶은 내용을 하나의 SQL문으로 기술하고 RDBMS에 전송한다.
- 원칙적으로는 문장 끝네 구분자로 ; 을 넣어야 한다.
- SQL은 목적에 따라 DDL, DML, DCL 로 분류할 수 있다.

## SQL 문과 종류

1. DDL ( Data Definition Language) 데이터 정의언어
2. DML ( Data Manipulation Language) 데이터 조작언어
3. DCL ( Data Control Language)  데이터 제어언어

## SQL 문과 종류

1. DDL 데이터 정의언어

   데이터를 저장하는 데이터베이스 및 테이블을 생성, 삭제하기 위한것

- CREATE : 데이터베이스나 테이블등을 작성
- DROP : 데이터베이스나 테이블등을 삭제
- ALTER : 데이터베이스나 테이블 등의 구성을 변경

## SQL 문과 종류

### 2. DML 데이터 조작언어

테이블의 행을 검색하거나 변경하기 위한것.

- SELECT : 테이블에서 행을 검색한다.
- INSERT : 테이블에 신규행을 등록한다.
- DELETE : 테이블에서 행을 삭제한다.

## SQL 문과 종류

### 3. DML 데이터 제어언어

데이터베이스에서 처리한 변경내용을 확정, 취소하기 위한것.

RDBMS 사용자에게 권한을 부여하기도 한다.

- COMMIT : 데이터베이스 변경내용을 확정한다.
- ROLLBACK : 데이터베이스 변경내용을 취소한다.
- GRANT : 사용자에게 처리권한을 부여한다.
- REVOKE : 사용자 처리 권한을 제거한다.

# SQL 문의 기본적인 작성 규칙

✓ **SQL문 마지막에 세미콜론 ; 을 붙인다.**

하나의 데이터베이스처리는 하나의 SQL문으로 이루어진다.

RDBMS도 SQL문을 하나씩 실행한다.

문장에는 문장 구분 문자로 세미콜론; 을 사용한다.

✓ **대문자 소문자 구분이 없다.**

✓ **상수 작성법에는 규칙이 있다.**

SQL문안에 직접 기술하는 문자열이나 날짜, 숫자등을 상수라고 한다.

- SQL문안에 문자열을 기술할 때는 'abc'와 같이 작은 따옴표 ' 로 문자열을 감싼다.
- 날짜에는 다양한 형식이 있기때문에 주의.
- 숫자를 기술할때는 어떠한 기호를 써서 표현할 필요가 없다. 숫자만 쓰면된다.

✓ **단어는 공백문자나 줄바꿈 문자로 구분한다.**

## 표준 SQL과 개별 구문

SQL 표준규격은 ANSI, IOS 같은 표준화 단체에 의해 수년에 한번정도 개정된다.

개정시 구문개정이나 신기능 추가가 이루어 진다.

ANSI가 처음 SQL 표준규격을 정한것은 1986년이다.

SQL 표준규격에는 모든 RDBMS가 이 규격을 따라야 한다는 강제성은 없다.

표준 SQL을 지원하는 RDBMS가 늘었지만 그래도 표준 SQL로 작성한 SQL 문을 실행하지 못하는 경우가 있다.

이런 경우에는 해당 RDBMS에서만 실행되는 SQL개별구문을 사용하여야한다.

1980-1990에는 표준SQL이 가지고 있는 기능이 빈약하여 실무에 이용하기 어려워 RDBMS 제조사들은 독자 기능을 추가할 수 밖에 없었다. 이중 매우 편리한것들은 표준 SQL로 채용된 것도 있다. 현재 표준 SQL은 여러번의 개벙을 거치며 다양한 기능을 충실히 구현하고 있다.

# I. 데이터베이스와 SQL – 4. 테이블 작성

## 데이터 베이스 작성 CREATE DATABASE문

- 테이블을 저장할 데이터베이스를 만든다.
- RDBMS에 데이터베이스 작성하는법

```
CREATE DATABASE <데이터베이스명>
```

# 테이블 작성 CREATE TABLE문

## 2. 테이블 작성

CREATE TABLE <테이블명> ( <열명1> <데이터형> <이 열의 제약>,
<열명2> <데이터형> <이 열의 제약>,
....
<이 테이블의 제약1>, <이 테이블의 제약2>, ..... );

```
shop=# CREATE TABLE Goods
shop-# (goods_id      CHAR(4)        NOT NULL,
shop(#  goods_name    VARCHAR(100)   NOT NULL,
shop(#  goods_classify VARCHAR(32)   NOT NULL,
shop(#  sell_price   INTEGER ,
shop(#  buy_price    INTEGER ,
shop(#  register_date       DATE ,
shop(#  PRIMARY KEY (goods_id));
```

## 명명규칙

- 데이터베이스나 테이블, 열 등의 이름에 사용할 문자는 영문자, 숫자,언더바만 가능
- 이름의 첫글자는 반드시 영문이어야 한다.
- 이름을 중복해서 사용해서는 안된다.

## 데이터형 지정

- **INTEGER 형**

  정수를 넣기 위한 데이터형이다. 소수는 넣을 수 없다.

- **CHAR형**

  CHARACTER 의 약자로 문자열을 넣기위한 데이터형

  CHAR(100)처럼 열안에 문자열을 최대길이를 지정가능.

  최대길이를 초과하는 문자열은 넣을 수 없다. 문자수, 또는 바이트기준임.

  고정문자열이라는 데이터가 저장됨.

  CHAR(8) -> 'abc_ _ _ _ _' 와같이 저장됨

## 데이터형 지정

- VARCHAR형

  가변 문자열. 문자수가 최대길이보다 작아도 공백으로 채우지 않는다.

  (오라클에서는 VARCHAR2 이다)

- DATE형

  날짜를 저장하는 날짜형 데이터형이다.

  (오라클의 DATE형은 데이터에 년월일뿐아니라 시분초까지 포함한다)

## 제약설정

- 제약이란 데이터형 외에도 열에 넣을 데이터에 제약이나 조건을 추가하는 기능.
- NOT NULL 제약 데이터가 없으면 에러가 난다.
- PRIMARY KEY (id) 주키제약

  주키, 기본키라고도하며 하나의 행을 특정 지을 수 있는 열을 의미한다.

  유일(유니크)하다라고도 표현한다.

# I. 데이터베이스와 SQL – 1-5 테이블 삭제와 변경

**테이블 삭제 DROP TABLE – 삭제한 데이터는 복구할 수 없다.!!**

**테이블정의변경 ALTER TABLE – 변경한 데이터는 원래대로 복구할 수 없다.!!**

```
DROP TABLE <테이블명>;
ALTER TABLE <테이블명> ADD COLUMN <열 정의>;     //열 추가
ALTER TABLE <테이블명> DROP COLUMN <열명>;       //열 삭제
```

**Goods 테이블에 goods_name_eng라는 길이 100인 가변 문자열을 저장하는 열 추가**

**추가한 goods_name_eng열 삭제**

```
shop=# ALTER TABLE Goods ADD COLUMN goods_name_eng VARCHAR(100);
ALTER TABLE
shop=# ALTER TABLE Goods DROP COLUMN goods_name_eng;
ALTER TABLE
shop=# DROP TABLE Goods;
DROP TABLE
```

## 테이블 수정

테이블이름을 잘못 만들었을때

아직 테이블에 데이터가 들어있지 않을때는 DROP 하고 CREATE 하면되나

대량의 데이터가 들어있을때 대부분의 데이터베이스가 테이블명을 변경할수있는

RENAME이라는 명령어를 제공하고 있다.

- Oracle, PostgreSQL - ALTER TABLE Gods RENAME TO Goods;
- DB2 - RENAME TABLE Gods TO Goods;
- SQL Server - sp_rename ‘Gods’, ‘Goods’;
- MySQL - RENAME TABLE Gods to Goods;

# II. 검색의 기본 – 1. SELECT 문의 기본

- 테이블에서 데이터를 선택하려면 SELECT문을 사용한다.
- 열에는 표시용 별명을 붙일 수 있다.
- SELECT 구에는 상수나 식을 쓸 수 있다.
- DISTINCT키워드를 지정하면 중복된 행을 제거할 수 있다.
- SQL문안에 메모를 남기기 위해서 주석을 작성할수있다
- WHERE 구를 사용해서 검색 조건에 부합하는 행을 테이블에서 선택할 수 있다.

## SELECT 필요한 데이터를 추출하는 쿼리

```
SELECT <열명>, ...
    FROM <테이블명>;
```

- SELECT 구에는 테이블에서 출력하고 싶은 열의 이름들을 기술
- FROM구에는 데이터를 꺼낼 대상 테이블의 이름을 지정
- Goods 테이블에서 세개 열을 선택하여 출력

```
SELECT goods_id, goods_name, buy_price
  FROM Goods;
```

- 이때 출력되는 열은 SELECT 구와 동일한 순서로 정렬됨

## 모든 열의 출력

```
SELECT  *
    FROM <테이블명>;
```

```
SELECT *
  FROM Goods;
```

- 이때 * 는 모든열을 의미한다.
- 단 별표를 사용하면 결과열의 정렬순서를 지정할 수 없다.

## 열에 별명을 부여하자.

- SQL문에서는 AS 키워드로 열에 별명을 부여할 수 있다.

```
SELECT
      goods_id    AS id,
      goods_name   AS name,
      buy_price AS price
 FROM Goods;
```

- 별명으로 한글을 사용할 수도있다. 큰따옴표 “ 를 사용한다.

```
SELECT
      goods_id    AS "상품ID",
      goods_name   AS "상품명",
      buy_price AS "매입단가"
 FROM Goods;
```

## 상수 출력

- SELECT 문에는 별명뿐아니라 문자열상수, 숫자상수,날짜 상수를 쓸 수 있다.
- SQL문안에 문자열이나 날짜상수를 적으려면 반드시 작은 따옴표 ' 를 사용.

```
SELECT '상품' AS munja, 38 AS num, '2009-02-24' AS nalja, goods_id, goods_name
    FROM Goods;
```

# 결과에서 중복 행을 제거

- Goods테이블에 어떤 상품 분류가 등록되어 있는지 알고 싶을때 중복을 제외한 데이터를 얻는것이 좋다.

| 상품ID | 상품명 | 상품분류 | 판매단가 | 등록일 |
|---|---|---|---|---|
| 0001 | 티셔츠 | 의류 | 1000 | 2015-04-05 |
| 0002 | 목걸이 | 악세서리 | 5000 | 2015-04-09 |
| 0003 | 벨트 | 악세서리 | 4000 | 2016-04-05 |
| 0004 | 구두 | 슈즈 | 12000 | 2017-04-09 |

중복행 제거

| 상품분류 |
|---|
| 의류 |
| 악세서리 |
| 슈즈 |

## 결과에서 중복 행을 제거

- SELECT 구에 DISTINCT라는 키워드를 사용

```
SELECT DISTINCT goods_classify
  FROM Goods;
```

- DISTINCT를 사용할때 NULL도 하나의 데이터로 인식된다.
- 복수의 행에 NULL이 있는 경우에 NULL도 하나의 데이터 그룹이 된다.
- DISTINCT는 복수의 열 앞에 놓을수도 있다.

```
SELECT DISTINCT goods_classify, register_date
  FROM Goods;
```

- DISTINCT 키워드는 열명 앞에만 놓일 수 있다. register_date, DISTINCT goods_classify와같이 쓸 수 없다

# WHERE 구를 사용한 행 선택

- SELECT문에서는 선택하고 싶은 행의 조건을 WHERE 구로 지정한다.

```
SELECT <열명>,...
  FROM  <tablename>
    WHERE <조건식>;
```

```
SELECT goods_name, goods_classify
 FROM Goods
WHERE goods_classify = '의류';
```

```
SELECT goods_name
 FROM Goods
WHERE goods_classify = '의류';
```

# 주석

```
SELECT DISTINCT goods_id, buy_price
-- 이 SELECT 문은 결과에서 중복을 제거한다
  FROM Goods;
```

```
/* 이 SELECT 문은 결과에서 중복을
   제거한다*/
SELECT DISTINCT goods_id, buy_price
  FROM Goods;
```

## 7. 산술연산자

- SQL문에 계산식을 사용할 수 있다.
- 주의 : NULL을 포함한 계산은 무조건 NULL이 된다.

```
shop=# SELECT goods_name, sell_price, sell_price*2 AS "sell_price X2" FROM Goods;
 goods_name | sell_price | sell_price X2
------------+------------+---------------
 T-shirt    |       1000 |          2000
 Punching   |        500 |          1000
 Y-shirt    |       4000 |          8000
 knife      |       3000 |          6000
 pot        |       6800 |         13600
 fork       |        500 |          1000
 cutboard   |        880 |          1760
 pen        |        100 |           200
```

| 의미 | 연산자 |
|---|---|
| 덧셈 | + |
| 뺄셈 | - |
| 곱셈 | * |
| 나눗셈 | / |

<산술 연산자의 종류>

## 8. 비교연산자

- 주의 : 이상, 이하의 연산자는 등호가 뒤에 와야 한다.
  NULL에 사용할 수 없다.
  문자열에 사용할 때 주의해야 한다.
  (ex. 문자열 '2'와 '10'의 크기를 비교하면
  '2'가 더 크다.)

```
shop=# SELECT goods_name, goods_classify FROM Goods WHERE sell_price=500;
 goods_name | goods_classify
------------+---------------
 Punching   | office
 fork       | kitchen
```

<정수형 데이터에 비교 연산자를 쓴 경우>

```
mydb=# SELECT goods_name, goods_classify, register_date FROM Goods
mydb-# WHERE register_date <= '2009-09-11';
 goods_name | goods_classify | register_date
------------+----------------+---------------
 Punching   | office         | 2009-09-11
 pot        | kitchen        | 2009-01-15
 cutboard   | kitchen        | 2009-04-28
```

| 의미 | 연산자 |
|---|---|
| 같다 | = |
| 같지 않다 | <> |
| 이상 | >= |
| 초과 | > |
| 이하 | <= |
| 미만 | < |

<논리 연산자의 종류>

NULL에 비교 연산자를 사용할 수 없다.

NULL이 아닌행을 선택하고 싶은 경우에는 IS NOT NULL을 사용한다.

```
SELECT goods_name
        FROM Goods
        WHERE by_price IS NOT NULL
```

# 논리 연산자

<논리 연산자의 종류>

| 우선순위 | 의미 | 연산자 |
|---|---|---|
| | 부정 | NOT |
| 1 | 그리고 | AND |
| 2 | 또는 | OR |

<논리 연산자(부정)>

```
SELECT goods_name, goods_classify, sell_price
  FROM Goods
 WHERE NOT sell_price >= 1000;
```

<논리 연산자(그리고)>

```
SELECT goods_name, buy_price
  FROM Goods
 WHERE goods_classify = '주방용품 '  AND  sell_price >= 3000;
```

<논리 연산자(또는)>

```
SELECT goods_name, buy_price
  FROM Goods
 WHERE goods_classify = '주방용품'  OR  sell_price >= 3000;
```

## 논리 연산자

- 괄호를 사용하면 우선순위가 높아진다.
- AND 와 OR가 같이있는경우 AND가 우선순위가 높다.

```
SELECT goods_name, goods_classify, register_date
 FROM Goods
WHERE goods_classify = '사무용품' AND register_date = '2009-09-11'
                                    OR register_date = '2009-09-20';
```

<괄호를 사용하지 않고 AND와 OR 연산자를 쓴 경우>

```
SELECT goods_name, goods_classify, register_date
 FROM Goods
WHERE goods_classify = '사무용품'  AND
          (   register_date = '2009-09-11' OR register_date = '2009-09-20');
```

<괄호를 사용하여 OR 연산자의 우선순위를 AND 연산자보다 높인 경우>

## MISSION1

1. Goods테이블에서 등록일이 2010년1월1일이후인 상품의 이름과 등록일을 출력한다.
2. Goods테이블에 대해 다음 세가지의 조건을 만족하는 데이터를출력.
   1) buy_price=null인 Goods테이블의 모든칼럼
   2) buy_price <> null 인 Goods테이블의 모든 컬럼
   3) buy_price > null 인 Goods테이블의 모든 컬럼
3. Goods테이블에서 판매단가가 매입단가보다 500원이상 높은 상품을 출력하는 SQL문을 2가지작성해보세요
4. Goods테이블에서 판매단가를 10%빼도 이익이 100원보다 높은 주방용품과 사무용품을 출력하는 SQL문을 작성해 보세요

   출력컬럼: goods_name, goods_classify, 판매단가를 10%뺀경우의 이익

# III. 집약과 정렬 – 1. 집약함수와 검색

## 집약함수

- SQL로 데이터에 어떠한 처리나 계산을 하려면 함수를 이용한다.
  - COUNT : 테이블 레코드 수(행수)를 계산한다.
  - SUM : 숫자열 데이터의 평균을 구한다.
  - AVG : 숫자열 데이터의 평균을 구한다.
  - MAX : 임의열이 가진 데이터의 최댓값을 구한다.
  - MIN : 임의열이 가진 데이터의 최솟값을 구한다.

# III. 집약과 정렬 – 1. 집약함수와 검색

- 전체 행수를 계산

```
SELECT COUNT(*)
  FROM Goods;
```

- NULL을 제외하고 행수를 계산

  대상 열을 한정해서 인수로 지정한다.

  CONT함수는 인수에 따라 처리결과가 달라지기때문에 주의가 필요하다.

```
SELECT COUNT(buy_price)
  FROM Goods;
```

# III. 집약과 정렬 – 1. 집약함수와 검색

- NULL을 포함한 열을 인수로 하는경우 COUNT * 과 COUNT(열)의 결과가 다르다.
- COUNT * 은 NULL을 포함하는 행수를, COUNT(열)은 NULL을 제외한 행수를 계산.

```
CREATE TABLE NullTbl
(col_1 INTEGER);

INSERT INTO NullTbl VALUES (NULL);
INSERT INTO NullTbl VALUES (NULL);
INSERT INTO NullTbl VALUES (NULL);

SELECT COUNT(*), COUNT(col_1)
  FROM NullTbl;
```

# III. 집약과 정렬 – 1. 집약함수와 검색

## 합계

```
SELECT SUM(sell_price)
  FROM Goods;
```

```
SELECT SUM(sell_price), SUM(buy_price)
  FROM Goods;
```

판매단가와 달리 매입단가에는 NULL을 포함한 행이 있다.

SUM함수는 NULL도 포함한 계산이 이루어지는데 ,

사칙연산에 NULL이 포함되는 경우 결과는 무조건 NULL이 되어야한다.

그렇다면 매입단가의 합계도 NULL이 되어야?

모든 집약함수는 열명을 인수로 가질때 계산전에 NULL을 제한한다.

따라서 NULL이 몇개가 있든 모두 무시되는 것이다. 그렇다고 0 과 같다고도 할 수 없다

집약합수는 NULL을 제외한다 단 COUNT(*)만 예외적으로 NULL을 포함한다.

# III. 집약과 정렬 – 1. 집약함수와 검색

## 평균값

```
SELECT AVG(sell_price)
  FROM Goods;
```

NULL을 포함한 매입단가의 평균값계산.

NULL을 0으로 간주해서 계산을 해야하는 경우가 있다.

```
SELECT AVG(sell_price),
AVG(buy_price)
  FROM Goods;
```

# III. 집약과 정렬 – 1. 집약함수와 검색

## 최대값, 최솟값

```
SELECT MAX(sell_price), MIN(buy_price)
  FROM Goods;
```

MAX/MIN함수는 SUM/AVG함수와 다른점이 한가지 있음.

SUM/AVG함수가 숫자형 열만 이용할 수 있다면,

MAX/MIN함수는 원칙상 어떤 데이터형도 사용할 수 있다.

```
SELECT MAX(register_date), MIN(register_date)
  FROM Goods;
```

# III. 집약과 정렬 – 1. 집약함수와 검색

집약함수 사용시 중복값 제외 DISTINCT 키워드 – 값의 종류를 세고 싶을때

```
SELECT COUNT(DISTINCT goods_classify)
  FROM Goods;
```

DISTINCT는 반드시 괄호안에 기술해야한다.

먼저 good_classify열의 중복값을 제외한 후에 행수를 계산하고 있기 때문이다.

```
SELECT DISTINCT COUNT(goods_classify)
  FROM Goods;
```

위와같이 괄호밖에 쓰면 먼저 goods_classify열의 행수를 센 후에 결과에서 중복값을 제외하게 되서 결과는 goods_classify열의 전체 행수 가 된다.

# III. 집약과 정렬 – 2. 그룹핑

테이블을 그룹으로 나눈다.

- GROUP BY 는 테이블을 나누어자른다.
- 집약함수와 GROUP BY구를 사용하면 테이블을 상품분류별 또는 등록일별등으로 나누어 집약할 수 있다.
- 집약 키에 NULL이 포함되면 집계결과에도 공란으로 표현된다.
- 집약함수와 GROUP BY 구를 사용할 때는 다음 네가지사항 주의

1. SELECT 구에 기술할 수 있는것이 제한된다.
2. GROUP BY구에는 SELECT구에서 사용한 별명을 사용할 수 없다.
3. GROUP BY구는 집약결과를 정렬하지 않는다.
4. WHERE 구에 집약함수를 쓸 수 없다.

# III. 집약과 정렬 – 2. 그룹핑

## GROUP BY

- 테이블을 몇개의 그룹으로 나누어 집약해 본다.

```
SELECT <열명1>, <열명2>, <열명3>, ...
        FROM <테이블명>
        WHERE
        GROUP BY <열명1>, <열명2>, <열명3>, ...;
```

```
SELECT goods_classify, COUNT(*)
  FROM Goods
 GROUP BY goods_classif
```

실행결과

```
goods_classify  | count
-----------------------------------
의류            | 2
사무용품        | 2
주방용품        | 4
```

# III. 집약과 정렬 – 2. 그룹핑

## GROUP BY

- 테이블을 몇개의 그룹으로 나누어 집약해 본다.

| goods_classify | goods_name | goods_id | sell_price | buy_price | register_date |
|---|---|---|---|---|---|
| 의류 | 티셔츠 | 0001 | 1000 | 500 | 2009-09-20 |
| 의류 | 와이셔츠 | 0003 | 4000 | 2800 | |
| 사무용품 | 펀칭키 | 0002 | 500 | 320 | 2009-09-20 |
| 사무용품 | 볼펜 | 0008 | 100 | | 2008-09-20 |
| 주방용품 | 식칼 | 0004 | 3000 | 2800 | 2009-09-20 |
| 주방용품 | 압력솥 | 0005 | 6800 | 5000 | 2009-007-20 |
| 주방용품 | 포크 | 0006 | 500 | | 2009-08-20 |

# III. 집약과 정렬 – 2. 그룹핑

## GROUP BY - NULL이 존재하는 경우

```
SELECT buy_price, COUNT(*)
  FROM Goods
 GROUP BY buy_price;
```

NULL이 집약 키행에 존재한다.

집약키에 null이 존재하는 경우

NULL이라는 별도그룹으로

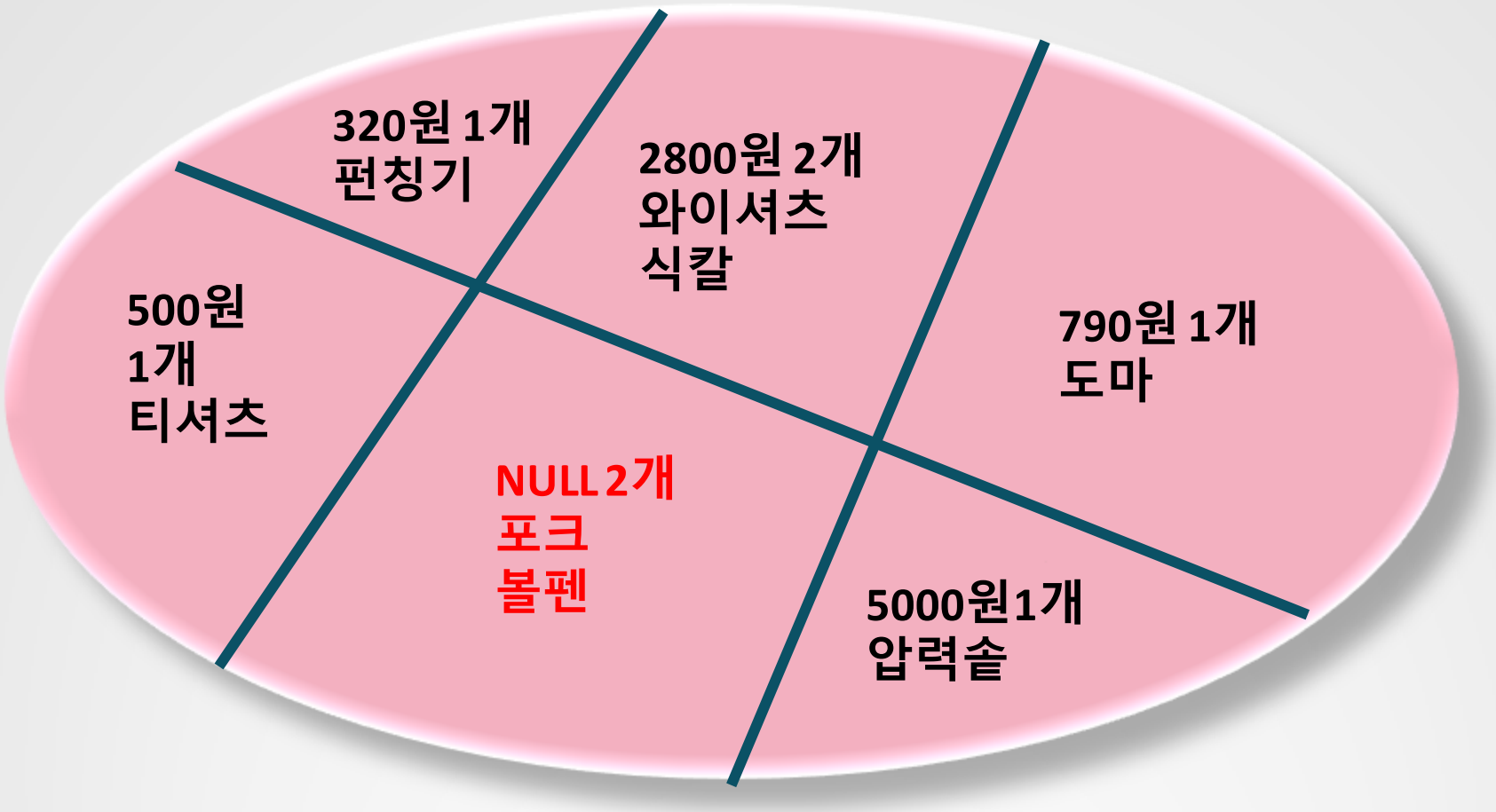
분류된다.

실행결과

| buy_price | count |
|---|---|
| | 2 |
| 320 | 1 |
| 500 | 1 |
| 5000 | 1 |
| 2800 | 2 |

# III. 집약과 정렬 – 2. 그룹핑

**GROUP BY – NULL이 포함된경우에도 불명 행 공란으로 존재한다.**



# III. 집약과 정렬 – 2. 그룹핑

WHERE 구를 사용한 경우 GROUP BY 동작

WHERE 구를 사용하여 집약하는 경우에는 WHERE 구에서 지정한 조건으로 레코드를 먼저 선택한 후에 집약이 이루어진다.

```
SELECT buy_price, COUNT(*)
  FROM Goods
 WHERE goods_classify = '의류'
 GROUP BY buy_price;
```

실행결과

```
buy_price  | count
-------------------------
     500   |  1
    2800   |  1
```

| goods_classify | goods_name | goods_id | sell_price | buy_price | register_date |
|---|---|---|---|---|---|
| 의류 | 티셔츠 | 0001 | 1000 | 500 | 2009-09-20 |
| 의류 | 와이셔츠 | 0003 | 4000 | 2800 | |

# III. 집약과 정렬 – 2. 그룹핑

- GROUP BY 와 WHERE를 병용한 경우 SELECT 문의 작성순서
- SELECT -> FROM -> WHERE -> GROUP BY
- GROUP BY 와 WHERE를 병용한 경우 SELECT 문의 실행순서
- FROM -> WHERE -> GROUP BY -> SELECT

영어적 사고방식이 익숙한 서구인이라면 이 실행순서가 자연스럽지만
우리나라 사람들은 앞에서부터 순서대로 읽고 해석하는것이
습관이 되어있기 때문에 주의가 필요하다.

# III. 집약과 정렬 – 2. 그룹핑

집약함수와 GROUP BY 구를 사용할 때 자주하는 실수

1. SELECT구에 필요없는 열을 쓴다.

- COUNT같은 집약함수를 사용한 경우는 SELECT구에 쓸 수 있는 요소가 매우 제한적이다.
- 실제로 집약함수를 사용시 다음 3가지만 SELECT구에 기술 가능
  1)상수
  2)집약함수
  3)GROUP BY 구에서 지정할 열(즉, 집약키)
- 이때 실수 집약키 이외의 열명을 SELECT 구에 사용하는것

```
SELECT goods_name, buy_price, COUNT(*)
  FROM Goods
 GROUP BY buy_price;
```

- ERROR:column goods.goods_name must appear in the GROUP BY clause or be used in an aggregate function

# III. 집약과 정렬 – 2. 그룹핑

## 1. 실수 - SELECT구에 필요없는 열을 쓴다.

```
SELECT goods_name, buy_price, COUNT(*)
  FROM Goods
 GROUP BY buy_price;
```

어떠한 키를 사용해서 그룹화하고 싶다는 것은, 결과에 출력되는 1행에 해당하는 단위도 해당 그룹이 된다.

매입단가로 그룹화하면 1행에 한개의 매입단가가 출력됨. 문제는 이 집약 키와 상품명이 반드시 일대일로 대응하지 않는다는 것.

매입단가가 2800원인 상품은 와이셔츠와 식칼인데 2800원 행에 어떤 상품을 표시해야하는가?. 어느쪽을 우선해서 표시한다는 규칙은 없다.

이러한 이유로 하나의 닙약 키에 대해 복수의 값이 존재하는 열을 SELECT 구에 포함하는 것은 논리적으로 불가능하다.

# III. 집약과 정렬 – 2. 그룹핑

집약함수와 GROUP BY 구를 사용할 때 자주하는 실수

2. GROP BY 구에 별명을 쓴다.

```
SELECT goods_classify AS sb, COUNT(*)
  FROM Goods
 GROUP BY sb;
```

실행되지 않는이유

DBMS내부 실행 순서상 SELECT 구가 GROUP BY 구보다 뒤에 실행되기 때문

때문에 GROUP BY가 실행되는 시점에서는 SELECT 구에서 부여한 별명을

DBMS가 아직 인식하지 못한다.

# III. 집약과 정렬 – 2. 그룹핑

집약함수와 GROUP BY 구를 사용할 때 자주하는 실수

3. GROUP BY 구는 결과의 순서를 정렬한다???

```
SELECT goods_classify
  FROM Goods
 GROUP BY goods_classify;
```

GROUP BY에의해 출력되는 수백 수천행의 결과는 순서없이 무작위로 출력된다.

일반적으로 SELECT 문 결과는 무작위로 정렬된다.

이정렬순서를 Sort 하기 위해서는 ORDER BY를 사용한다.

# III. 집약과 정렬 – 2. 그룹핑

## 집약함수와 GROUP BY 구를 사용할 때 자주하는 실수

### 4. WHERE 구에 집약 함수를 사용한다.

```
SELECT goods_classify, COUNT(*)
  FROM Goods
 WHERE COUNT(*) = 2
 GROUP BY goods_classify;
```

- ERROR : aggregates not allowed in WHERE clause

COUNT등의 집약함수를 사용할 수 있는 부분은 SELECT 구와 HAVING 구 ORDER BY 뿐이다.

# III. 집약과 정렬 – 2. 그룹핑

여기서 잠깐!!

DISTINCT 와 GROUP BY

```
SELECT DISTINCT goods_classify
  FROM Goods;

SELECT goods_classify
   FROM Goods
   GROUP BY goods_classify;
```

두개의 SELECT 구문은 동일한 결과를 출력함.

NULL을 하나의 행으로 간주하는것도 동일하며, 복수열을 사용하는 경우 결과도 같다.

결과내용도 같고 실행속도도 거의 동일하다.

선택결과에서 중복을 제외하고 싶다 : DISTINCT

집약한 결과를 구하고 싶다 : GROUP BY를 사용

# III. 집약과 정렬 – 3. 집약한 결과에 조건을 지정

## HAVING

**SELECT -> FROM -> WHERE -> GROUP BY -> HAVING**

```
shop=# SELECT goods_classification, AVG(sell_price)
shop-# FROM Goods
shop-# GROUP BY goods_classification
shop-# HAVING AVG(sell_price)>=2500;
 goods_classification |          avg
----------------------+-----------------------
 의류                 | 2500.0000000000000000
 주방용품             | 2795.0000000000000000
(2개 행)
```

# III. 집약과 정렬 – 3. 집약한 결과에 조건을 지정

## HAVING

**SELECT -> FROM -> WHERE -> GROUP BY -> HAVING**

```
shop=# SELECT goods_classification, COUNT(*)
shop-# FROM Goods
shop-# GROUP BY goods_classification
shop-# HAVING goods_classification='의류';
 goods_classification | count
----------------------+-------
 의류                 |     2
(1개 행)

shop=# SELECT goods_classification, COUNT(*)
shop-# FROM Goods
shop-# WHERE goods_classification='의류'
shop-# GROUP BY goods_classification;
 goods_classification | count
----------------------+-------
 의류                 |     2
(1개 행)
```

# III. 집약과 정렬 – 3. 집약한 결과에 조건을 지정

HAVING 보다 WHERE 에 쓰는것이 좋은 조건 – 집약키에대한 조건

집약키에 대한 조건은 HAVING구에도 WHERE구에도 쓸 수 있다.

```
SELECT goods_classify, COUNT(*)
  FROM Goods
 GROUP BY goods_classify
HAVING goods_classify = '의류';
```

```
SELECT goods_classify, COUNT(*)
  FROM Goods
WHERE goods_classify = '의류'
 GROUP BY goods_classify;
```

그러나 집약키에 대한조건은 WHERE절에 쓰는것이좋다.

이유1. HAVING구는 그룹에 대한 조건을 지정

WHERE 은 행에 대한 조건

단순히 행에 대한 조건은 WHERE절에 씀으로써 상호 기능을

확실하게 분리해서 읽기 쉬운 코드를 만든다.

# III. 집약과 정렬 – 3. 집약한 결과에 조건을 지정

HAVING 보다 WHERE 에 쓰는것이 좋은 조건 – 집약키에대한 조건

2. DBMS 내부처리때문

일반적으로 HAVING구보다 WHERE 구에 조건을 기술하는것이 처리속도가 빠르며

결과를 반환하는 시간도 짧다.

WHERE구를 사용하여 조건을 지정하면 소트전에 행을 걸러내기때문에

소트 대상행수를 줄일 수 있다.

HAVING구는 소트가 끝나고 그룹화되고나서 실행되기 때문에 WHERE보다 행수가많다.

또 WHERE구가 속도에서 유리한 이유는 WHERE에서 인덱스로 고속처리하기때문.

# III. 집약과 정렬 – 4. 검색결과를 재정렬한다.

```
SELECT <열명1>, <열명2>, <열명3>, ...
FROM <테이블명>
ORDER BY <재정렬 기준 열1>, <재정렬 기준 열2>, ...
```

▶ ORDER BY 구는 항상 SELECT 문 제일 마지막에 기술한다.

SELECT -> FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY

▶ 복수의 소트 키를 지정할 수 있다.
왼쪽 키부터 우선적으로 사용

```
SELECT goods_id, goods_name, sell_price, buy_price
  FROM Goods
ORDER BY sell_price ASC;
```

판매단가가 낮은순 오름차순

```
SELECT goods_id, goods_name, sell_price, buy_price
  FROM Goods
ORDER BY sell_price DESC;
```

판매단가가 높은순 내림차순

# III. 집약과 정렬 – 4. 검색결과를 재정렬한다.

## 복수의 SORT

```
SELECT goods_id, goods_name, sell_price, buy_price
  FROM Goods
ORDER BY sell_price, goods_id;
```

NULL을 포함하는 열을 소트키로 지정한 경우 NULL은 제일 앞 또는 제일 뒤에 모아서 표시된다.

```
SELECT goods_id, goods_name, sell_price, buy_price
  FROM Goods
ORDER BY buy_price;
```

```
SELECT goods_id AS id, goods_name, sell_price AS ht, buy_price
  FROM Goods
ORDER BY ht, id;
```

## Mission2

1. sell_price합계가 buy_price합계보다 1.5배이상 큰 상품분류를 구하는 SELECT 문을 작성해보세요

2. [이전에 Goods 테이블에서 모든 레코드를 선택하는 SELECT문을 사용하여 결과를 취득했다. 당시 ORDER BY구를 사용하여 순서를 지정했는지 잊어버렸다.]
실행결과를 참고로 해서 어떤 ORDER BY구가 적용되었는지 SQL문 작성해보세요.

| goods_id | goods_name | goods_classify | sell_price | buy_price | register_date |
|---|---|---|---|---|---|
| 0008 | 볼펜 | 사무용품 | 100 | | 2009-11-11 |
| 0006 | 포크 | 주방용품 | 500 | | 2009-09-20 |
| 0001 | 티셔츠 | 의류 | 1000 | 500 | 2009-09-20 |
| 0004 | 식칼 | 주방용품 | 3000 | 2800 | 2009-09-20 |
| 0002 | 펀칭기 | 사무용품 | 500 | 320 | 2009-09-11 |
| 0005 | 압력솥 | 주방용품 | 6800 | 5000 | 2009-01-15 |
| 0007 | 도마 | 주방용품 | 880 | 790 | 2008-04-28 |
| 0003 | 와이셔츠 | 의류 | 4000 | 2800 | |

# IV. 데이터 갱신 – 1. 데이터등록

● INSERT문 기본 구문

```
INSERT INTO <테이블명> (<열명1>, <열명2>,...)     VALUES (<값1>, <값2>,...);
```

## 1. 테이블 만들기

```
CREATE TABLE GoodsIns
(goods_id    CHAR(4)      NOT NULL,
 goods_name    VARCHAR(100) NOT NULL,
 goods_classify VARCHAR(32)  NOT NULL,
 sell_price  INTEGER      DEFAULT 0,
 buy_price  INTEGER      ,
 register_date      DATE         ,
 PRIMARY KEY (goods_id));
```

## 테이블에 1행 데이터를 등록

```
INSERT INTO GoodsIns
(goods_id, goods_name, goods_classify,
sell_price, buy_price, register_date)
VALUES
('0001', '티셔츠' ,'의류', 1000, 500, '2009-09-20');
```

# IV. 데이터 갱신 – 1. 데이터등록

- 원칙적으로 INSERT문은 1회실행에 1회 입력한다.
- 복수행 INSERT

  사실 많은 RDBMS에서는 1회의 복수 행을 INSERT하는것이 가능하다.

  복수행 INSERT라고 한다.

  복수행 INSERT는 INSERT문 작성내용에 실수가 있거나 부정확한 데이터를 삽입하려고 할때 복수행은 어느행 , 어느지점에서 에러가 발생했는지 찾아내기어렵다.

  복수행 INSERT는 DB2, SQL Server, PostgreSQL, MySQL에서 사용가능. Oracle안됨

# IV. 데이터 갱신 – 1. 데이터등록

● 일반 INSERT

```
INSERT INTO GoodsIns VALUES ('0002', '펀칭기', '사무용품', 500, 320, '2009-09-11');
INSERT INTO GoodsIns VALUES ('0003', '와이셔츠', '의류', 4000, 2800, NULL);
INSERT INTO GoodsIns VALUES ('0004', '식칼', '주방용품', 3000, 2800, '2009-09-20');
```

● 복수행 INSERT

```
INSERT INTO GoodsIns VALUES
        ('0002', '펀칭기', '사무용품', 500, 320, '2009-09-11')
        ('0003', '와이셔츠', '의류', 4000, 2800, NULL),
        ('0004', '식칼', '주방용품', 3000, 2800, '2009-09-20');
```

## 열 리스트 생략

● 테이블의 모든 열에 대해 INSERT문을 실행하는경우 테이블명 뒤에 기술하는 열 리스트 생략 가능.

●VALUE 구 값이 왼쪽 부터 순서대로 각열에 할당.

열 리스트 있음

```
INSERT INTO GoodsIns (goods_id, goods_name, goods_classify, sell_price, buy_price, register_date)
                    VALUES ('0005', '압력솥', '주방용품', 6800, 5000, '2009-01-15');
```

열 리스트 없음

```
INSERT INTO GoodsIns VALUES ('0005', '압력솥', '주방용품', 6800, 5000, '2009-01-15');
```

## NULL 값의 삽입

●INSERT문으로 열에 NULL을 할당하고 싶은 경우,해당열이 NOT NULL제약이 아닌경우

```
INSERT INTO GoodsIns
(goods_id, goods_name, goods_classify, sell_price, buy_price, register_date)
VALUES
('0006', '포크', '주방용품', 500, NULL, '2009-09-20');
```

●INSERT뿐만아니라 DELETE, UPDATE 등의 갱신문에 대하여 SQL문에 에러처리가 되면 테이블내의 데이터에는 아무 변동이 없다.

●이전에 이미 등록되어 있던 데이터는 사라지거나 망가지지 않는다.

## 기본값 삽입

●테이블 열에는 초기값을 설정할 수 있다.

1. 명시적 방법

VALUE구에 DEFAULT 키워드를 지정한다.

```
CREATE TABLE GoodsIns
(goods_id     CHAR(4)      NOT NULL,
 goods_name    VARCHAR(100) NOT NULL,
 goods_classify  VARCHAR(32)  NOT NULL,
 sell_price  INTEGER      DEFAULT 0,
 buy_price  INTEGER      ,
 register_date      DATE         ,
 PRIMARY KEY (goods_id));
```

```
INSERT INTO GoodsIns
(goods_id, goods_name, goods_classify, sell_price, buy_price, register_date)
VALUES
 ('0007', '도마', '주방용품', DEFAULT, 790, '2009-04-28');

-- 결과 확인
SELECT * FROM GoodsIns WHERE goods_id = '0007';
```

## 2. 암묵적 기본값 삽입

DEFAULT 키워드 없이 기본값이 설정되어있는열을 열 리스트 및 VALUE구에서 생략하기만 하면 기본값이 할당된다.

기본값이 설정되있지 않은 열의 경우에는 NULL이 할당된다.

NOT NULL인 경우에는 에러처리가 된다.

```
-- buy_price 열(제약 없음)을 생략: 'NULL'이 된다
INSERT INTO GoodsIns (goods_id, goods_name, goods_classify, sell_price,
register_date) VALUES ('0008', '볼펜', '사무용품', 100, '2009-11-11');

-- 결과 확인
SELECT * FROM GoodsIns WHERE goods_id = '0008';

-- goods_name 열(NOT NULL제약)을 생략: 에러!
INSERT INTO GoodsIns (goods_id, goods_c
```

다른 테이블에서 데이터를 복사하는 방법

1. 샘플테이블을 만든다

```
-- 데이터 등록 대상인 상품 복사 테이블
CREATE TABLE GoodsCopy
(goods_id    CHAR(4)      NOT NULL,
 goods_name    VARCHAR(100) NOT NULL,
 goods_classify VARCHAR(32)  NOT NULL,
 sell_price  INTEGER      ,
 buy_price  INTEGER      ,
 register_date     DATE         ,
 PRIMARY KEY (goods_id));
```

2. GoodsCopy테이블에
   Goods테이블데이터를 삽입하여 보자

```
-- 상품 테이블 데이터를 상품 복사 테이블로 복사
INSERT INTO GoodsCopy (goods_id, goods_name, goods_classify, sell_price, buy_price, register_date)
SELECT goods_id, goods_name, goods_classify, sell_price, buy_price, register_date
  FROM Goods;

-- 결과 확인
SELECT * FROM GoodsCopy;
```

# IV. 데이터 갱신 – 2. 데이터 삭제

DROP TABLE 문과 DELETE 문

데이터 삭제 방법은 2가지가 있다.

1. DROP TABLE문을 사용하여 테이블 자체를 삭제한다.
2. DELETE 문을 사용해서 테이블은 남겨두고 테이블내에 있는 모든 행을 삭제한다.

# IV. 데이터 갱신 – 2. 데이터 삭제

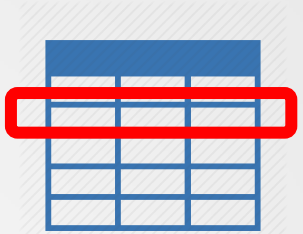
## DELETE 문

```
DELETE FROM <테이블명>;
```

```
DELETE FROM goods ;
```

DELETE문의 삭제대상은 레코드단위 이기때문에

DELETE문으로 일부 컬럼만 삭제하는것은 불가능하다.



# IV. 데이터 갱신 – 3. 데이터 삭제

DELETE 문

일부 행을 삭제할 때에는 WHERE 구로 대상 행의 조건을 기술한다.

```
DELETE FROM <테이블명>
   WHERE <조건> ;
```

```
DELETE FROM goods
 WHERE sell_price >= 4000; ;
```

TRUNCATE <테이블명>

반드시 테이블의 모든 데이터를 삭제한다.세밀한 제어는 되지 않지만

DELETE보다 고속으로 데이터를 삭제한다.(MySQL,ORACLE,SQL Server,PostgreSQL)

TRUNCATE를 사용할때는 DBMS의 매뉴얼에따라 주의하여 사용한다.

# IV. 데이터 갱신 – 3. 데이터 갱신

## UPDATE문

테이블 데이터를 변경함

```
UPDATE <테이블명>
  SET <열명>=<값> ;
```

등록일을 모두 2009년 10월 10로 변경

```
UPDATE Goods
  SET register_date = '2009-10-10';

-- 변경 내용 확인
SELECT * FROM Goods ORDER BY goods_id;
```

# IV. 데이터 갱신 – 3. 데이터 갱신

## 조건을 지정한 UPDATE문

```
UPDATE Goods
  SET sell_price = sell_price * 10
 WHERE goods_classify = '주방용품';

-- 변경 내용 확인
SELECT * FROM Goods ORDER BY goods_id;
```

## NULL로 갱신

```
UPDATE Goods
  SET register_date = NULL
 WHERE goods_id = '0008';

-- 변경 내용 확인
SELECT * FROM Goods ORDER BY goods_id;
```

# IV. 데이터 갱신 – 3. 데이터 갱신

## 복수의 열 갱신

주방용품 판매단가를 변경하고 동시에 매입단가도 변경하려면?

1) 번째 방법

```
-- 1회 UPDATE로 1열만 갱신한다
UPDATE Goods
   SET sell_price = sell_price * 10
 WHERE goods_classify = '주방용품';

UPDATE Goods
   SET buy_price = buy_price / 2
 WHERE goods_classify = '주방용품';
```

# IV. 데이터 갱신 – 3. 데이터 갱신

## 복수의 열 갱신

주방용품 판매단가를 변경하고 동시에 매입단가도 변경하려면?

2) 번째 방법 – 열을 쉼표로 구분해서 나열 한다.

```
UPDATE Goods
  SET sell_price = sell_price * 10,
      buy_price = buy_price / 2
 WHERE goods_classify = '주방용품';

-- 변경 내용 확인
SELECT * FROM Goods ORDER BY goods_id;
```

# IV. 데이터 갱신 – 3. 데이터 갱신

## 복수의 열 갱신

주방용품 판매단가를 변경하고 동시에 매입단가도 변경하려면?

3) 번째 방법 – 열을 괄호로 감싸서 리스트로 표현한다. DB2, PostgreSQL만가능

```
UPDATE Goods
  SET (sell_price, buy_price) = (sell_price * 10, buy_price / 2)
 WHERE goods_classify = '주방용품';

-- 변경 내용 확인
SELECT * FROM Goods ORDER BY goods_id;
```

# IV. 데이터 갱신 – 4. 트랜잭션

- 트랜잭션이란 세트로 실행되야 할 하나 이상의 변경 처리 집합이다.
- 트랜잭션을 사용하면 데이터베이스의 변경처리 확정 및 취소등을 관리할 수 있다.
- 트랜잭션 처리를 종료하는 명령으로 COMMIT 과 ROLLBACK 두가지가 있다.
- DBMS 트랜잭션에는 원자성, 일관성, 독립성, 지속성으로 이루어지는 4가지 규칙이 있다. 이를 ACID 특성이라고 한다.

# IV. 데이터 갱신 – 4. 트랜잭션

- 트랜잭션이란 무엇인가?

DBMS 에서의 Transaction 은 데이터베이스에서 이루어지는

하나 이상의 갱신처리를 모아서 부르는 명칭.

트랜잭션이란, 세트로 실행해야할 하나 이상의 갱신 처리 집합이다.

예) 와이셔츠 판매단가를 1000원 내리고 티셔츠 판매단가를 1000원 올리기로했어요. 변경내용을 데이터베이스에 반영해 주세요

```
UPDATE Goods
  SET sell_price = sell_price – 1000
  WHERE goods_name = '와이셔츠';
```

```
UPDATE Goods
  SET sell_price = sell_price + 1000
  WHERE goods_name = '티셔츠';
```

위의 두처리는 반드시 세트로 실행되어야 함.

이렇게 세트로실행되어야 할 갱신집합은 반드시 트랜잭션으로 모아서 취급.

# IV. 데이터 갱신 – 4. 트랜잭션

## 트랜잭션 구문

```
BEGIN TRANSACTION;          -- 트랜잭션 게시문
 DML문1;                    -- INSERT/UPDATE/DELETE문
 DML문2;
        …
 COMMIT; OR ROLLBACK;       -- 처리 확정 OR 처리 취소
```

| RDBMS | 트랜잭션 게시문 |
|---|---|
| SQL Server, PostgreSQL | BEGIN TRANSACTION |
| MySQL | START TRANSACTION |
| Oracle, DB2 | 없음 |

# IV. 데이터 갱신 – 4. 트랜잭션

## 트랜잭션 구문

Oracle,DB2

```
UPDATE Goods
      SET sell_price = sell_price - 1000
    WHERE goods_name = '와이셔츠';

UPDATE Goods
      SET sell_price = sell_price + 1000
    WHERE goods_name = '티셔츠';

COMMIT;
```

MySQL

```
START TRANSACTION;

   UPDATE Goods
      SET sell_price = sell_price - 1000
    WHERE goods_name = '와이셔츠';

   UPDATE Goods
      SET sell_price = sell_price + 1000
    WHERE goods_name = '티셔츠';

COMMIT;
```

SQL Server,
PostgreSQL

```
BEGIN TRANSACTION;

   UPDATE Goods
      SET sell_price = sell_price - 1000
    WHERE goods_name = '와이셔츠';

   UPDATE Goods
      SET sell_price = sell_price + 1000
    WHERE goods_name = '티셔츠';

COMMIT;
```

# IV. 데이터 갱신 – 4. 트랜잭션

## COMMIT – 처리확정

- COMMIT 이란 트랜잭션에 의한 변경내용을 모두 반영한 후에 트랜잭션을 종료하는 명령.
- 한번 커밋하면 더 이상 트랜잭션 개시 이전으로 돌아갈 수 없다.
- 최악의 경우 데이터를 복원 할 수 없는 경우도 있으니 주의가 필요하다.

# IV. 데이터 갱신 – 4. 트랜잭션

## ROLLBACK – 처리 취소

- ROLLBACK은 트랜잭션에 의해 처리된 변경 내용을 모두 취소하고 트랜잭션을 종료하는 명령.
- 저장하지 않고 종료.
- 롤백하면 데이터베이스 상태가 트랜잭션 개시전으로 돌아간다.

```
START TRANSACTION;

UPDATE Goods
      SET sell_price = sell_price - 1000
    WHERE goods_name = '와이셔츠';

UPDATE Goods
      SET sell_price = sell_price + 1000
    WHERE goods_name = '티셔츠';

ROLLBACK;
```

MySQL

## 트랜잭션은 언제 시작되는가?

- 트랜잭션개시문은 존재하나 실제로는 대부분 제품에서 트랜잭션 개시 명령을 사용하지 않는다.
- 대부분의 경우 데이터베이스 접속시점에 트랜잭션이 암묵적으로 개시된다.
- Oracle은 두가지 패턴을 사용하여 트랜잭션을 처리한다.

1. 하나의 SQL문에 하나의 트랜잭션 규칙적용 . 자동 커밋모드
2. 사용자가 COMMIT 또는 ROLLBACK을 실행하기까지가 하나의 트랜잭션으로간주.

- 일반적인 DBMS에서는 둘다 선택할 수 있다.
- SQL Server, PostgreSQL, MySQL등은 1번 자동커밋모드
- Oracle에서는 2번이 기본설정모드임

# IV. 데이터 갱신 – 4. 트랜잭션

## ACID 특성

- DBMS 트랜잭션에는 지켜야할 네가지 약속이 표준규격으로 정해져 있다.

1. Atomicity 원자성
2. Consistency 일관성
3. Isolation 독립성
4. Durability 지속성

# IV. 데이터 갱신 – 4. 트랜잭션

## ACID 특성

1. Atomicity 원자성

- 트랜잭션이 끝난 시점에 모든 갱신 처리가 실행된 상태 또는 모두 실행되지 않은 상태로 종료되는 것을 보증하는 성질.
- All or Nothing
- 예로 와이셔츠값은 내렸는데 티셔츠값은 올리지 않은 상태로 끝나는 트랜잭션은 절대로 없다.
- 트랜잭션 종료상태는 둘다실행 COMMIT 혹은 둘다 실행안됨 ROLLBACK 양자택일 이어야 한다.

# IV. 데이터 갱신 – 4. 트랜잭션

## ACID 특성

### 2. Consistency 일관성

- 트랜잭션에 포함되는 처리는 데이터베이스에 미리 설정된 제약을 지켜야 한다는 성질.
- NOT NULL제약을 NULL로 갱신하거나 주키제약 위반 레코드를 삽입하는 SQL문은 에러로 처리된다.

# IV. 데이터 갱신 – 4. 트랜잭션

## ACID 특성

### 3. Isolation 독립성

- 트랜잭션 상호간에 서로 간섭하지 않음을 보증하는 성질
- 어떤 트랜잭션에 의한 변경은 트랜잭션 종료시까지 다른 트랜잭션으로부터 은폐된다.
- 어떤 트랜잭션이 테이블에 레코드를 추가했다고 해도 커밋하기 전까지는 다른 트랜잭션이 추가한 레코드를 인식할 수 없다.

# IV. 데이터 갱신 – 4. 트랜잭션

## ACID 특성

### 4. Durability 지속성, 영속성

- 트랜잭션이 종료되면 해당 시점의 데이터 상태가 저장되는 것을 보증하는 성질.
- 시스템에 장애가 발생해서 데이터가 망가진 경우라도 데이터베이스는 어떠한 방법으로든 복구 수단을 가지고 제공해야 한다.
- 영속성 보증방법은 구현 방법에 따라 다른데 일반적으로 트랜잭션 실행 기록을 디스크등에 기록해 두었다가 (이를 로그라함) 장애가 발생한 경우에 이 로그를 사용하여 장애 이전 상태로 복원하는 방법이다.

## MySQL의 원격 접속

1. 원격접속 유저아이디 만들기

```
CREATE USER kim ;
```

2. 모든 원격 클라이언트에게 접속 허가

```
GRANT ALL PRIVILEGES ON *.* TO '{계정명}'@'%' IDENTIFIED BY '{비밀번호}' WITH GRANT OPTION;
```

```
GRANT ALL PRIVILEGES ON *.* TO ‘kim'@'%' IDENTIFIED BY ‘1111' WITH GRANT OPTION;
```

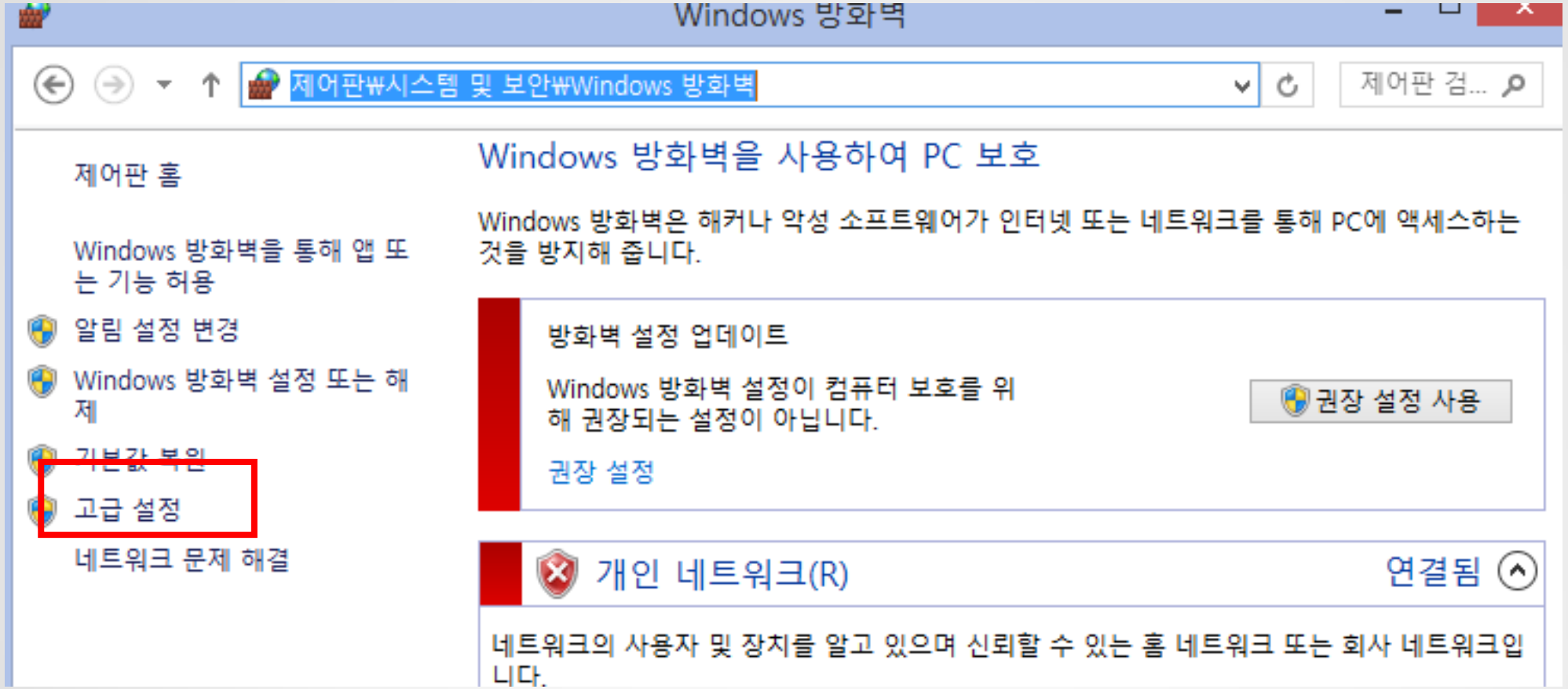
3. 특정 원격 클라이언트에게만 접속 허가

```
GRANT ALL PRIVILEGES ON *.* TO '{원격지접속용계정명}'@'{원격지IP주소}'
                                IDENTIFIED BY '{원격지에서접속시비밀번호}' WITH GRANT OPTION;
```

```
GRANT ALL PRIVILEGES ON *.* TO ‘kim'@‘192.168.1.1' IDENTIFIED BY ‘1111' WITH GRANT OPTION;
```

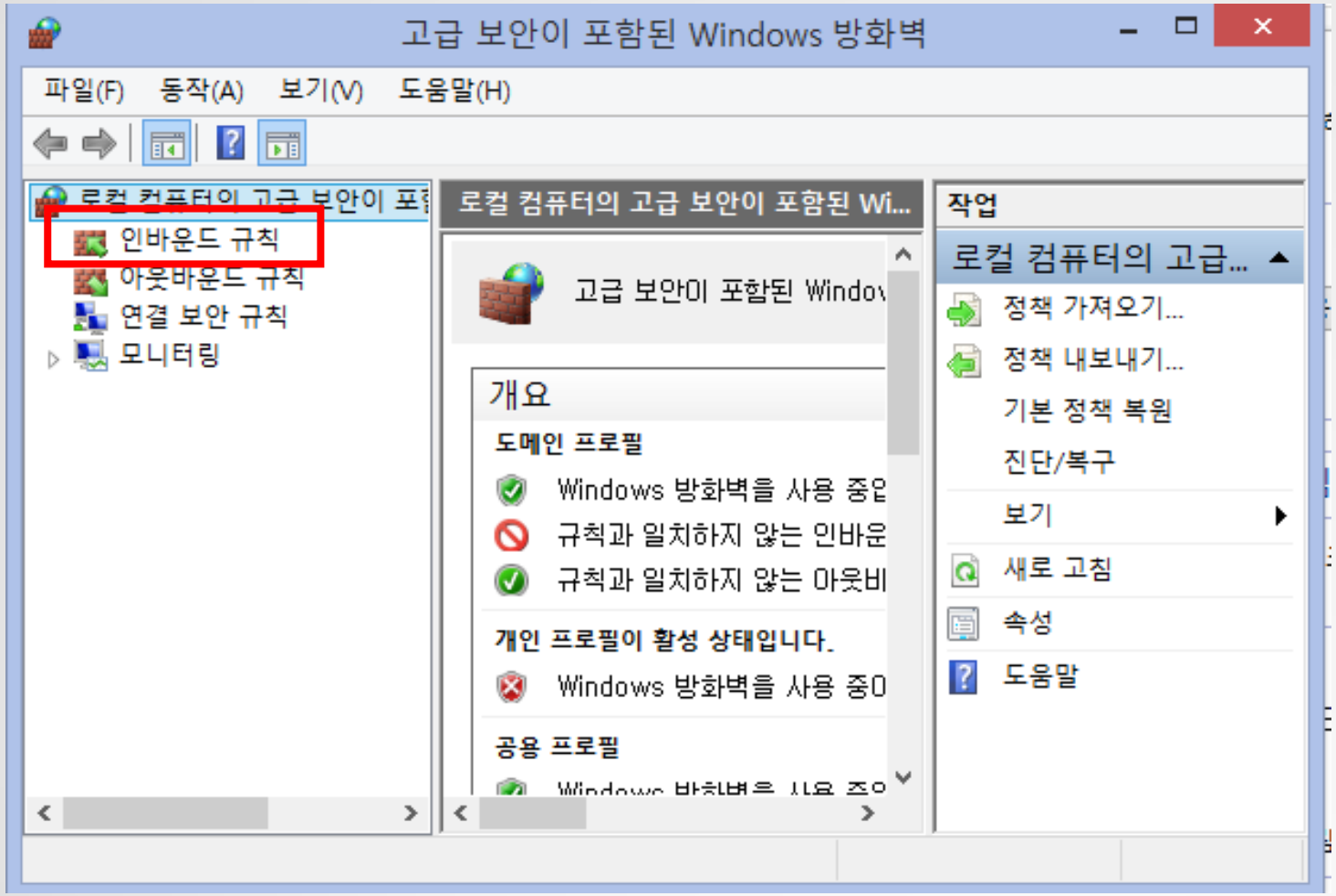
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정**



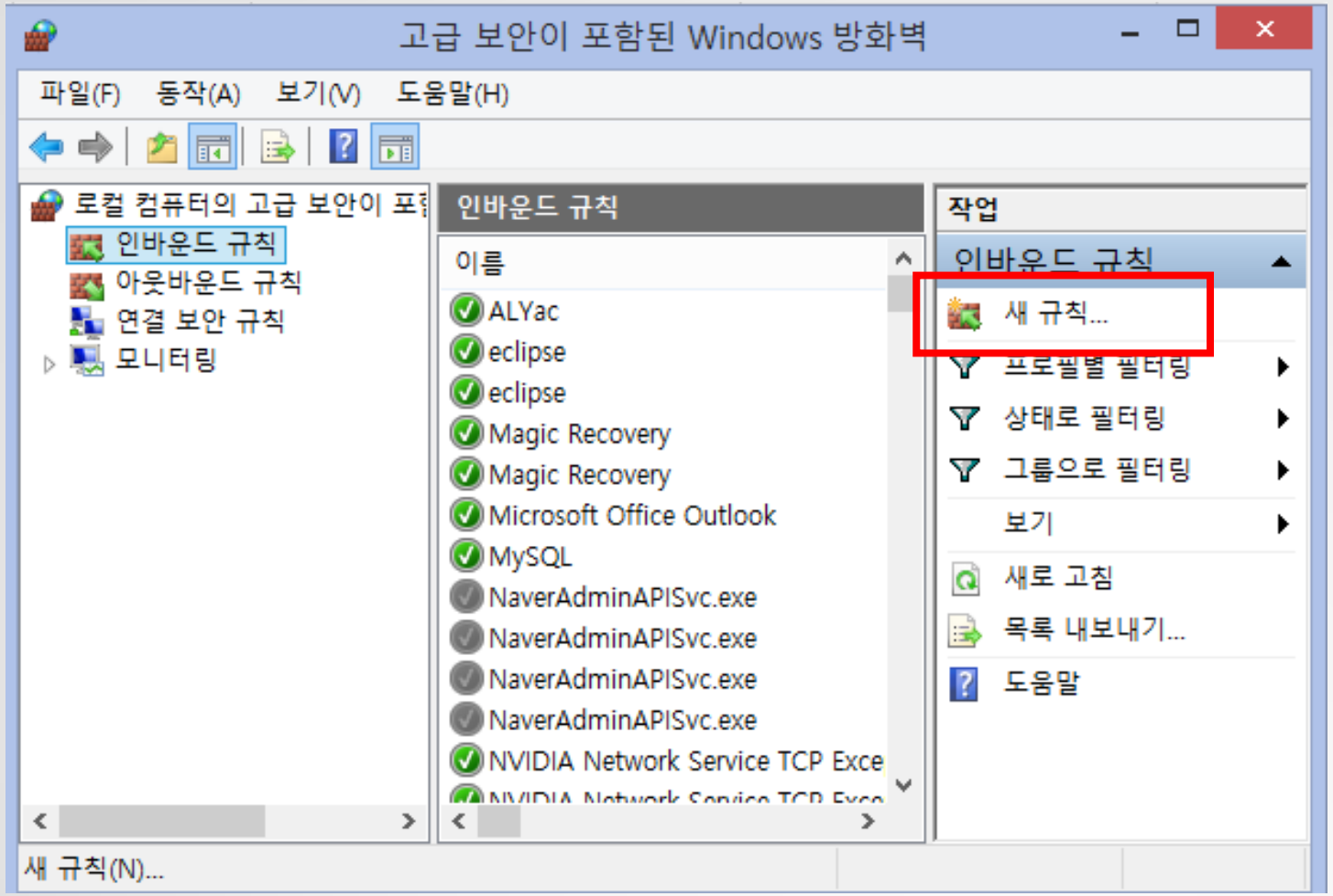
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙**



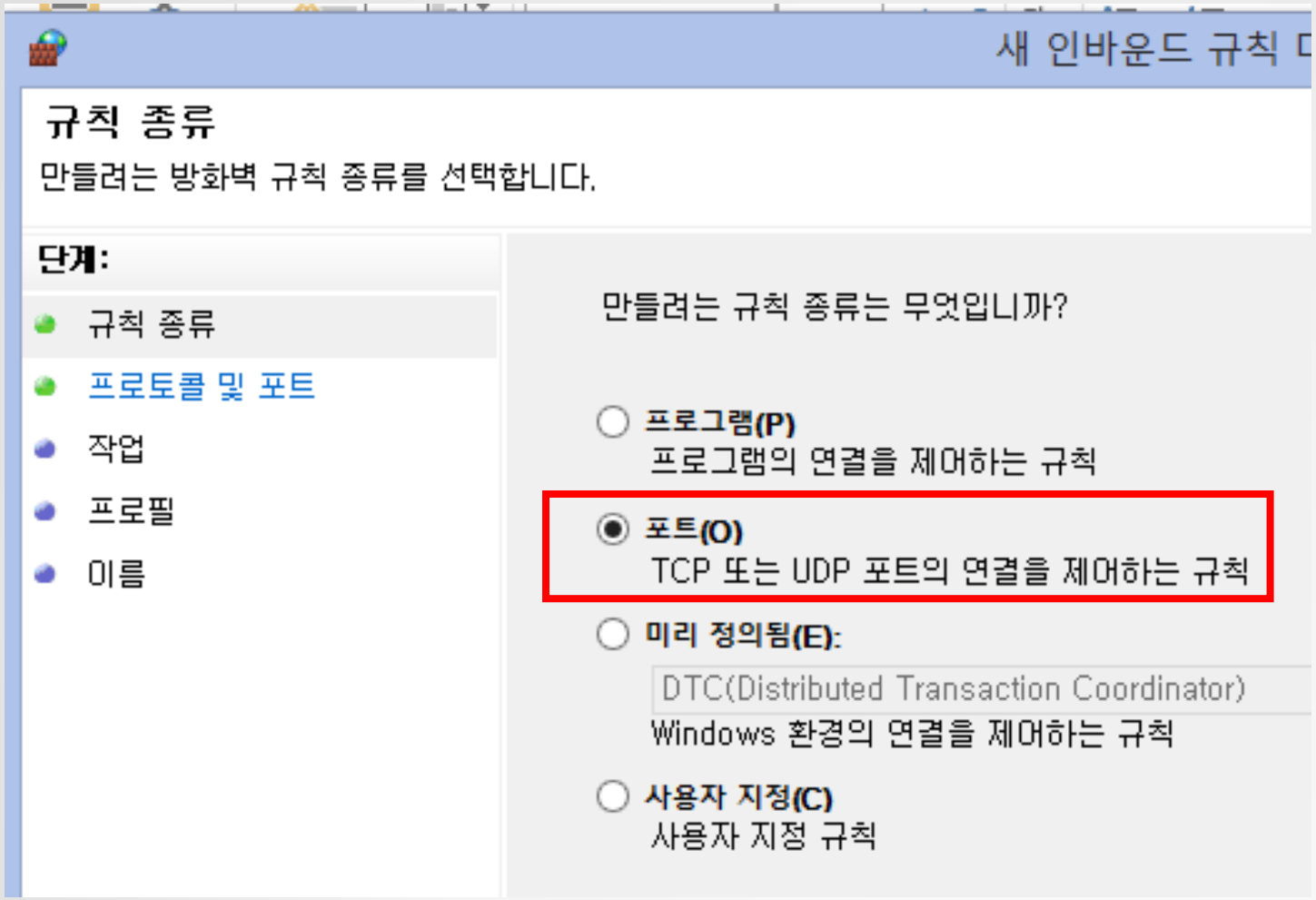
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙>>새규칙**



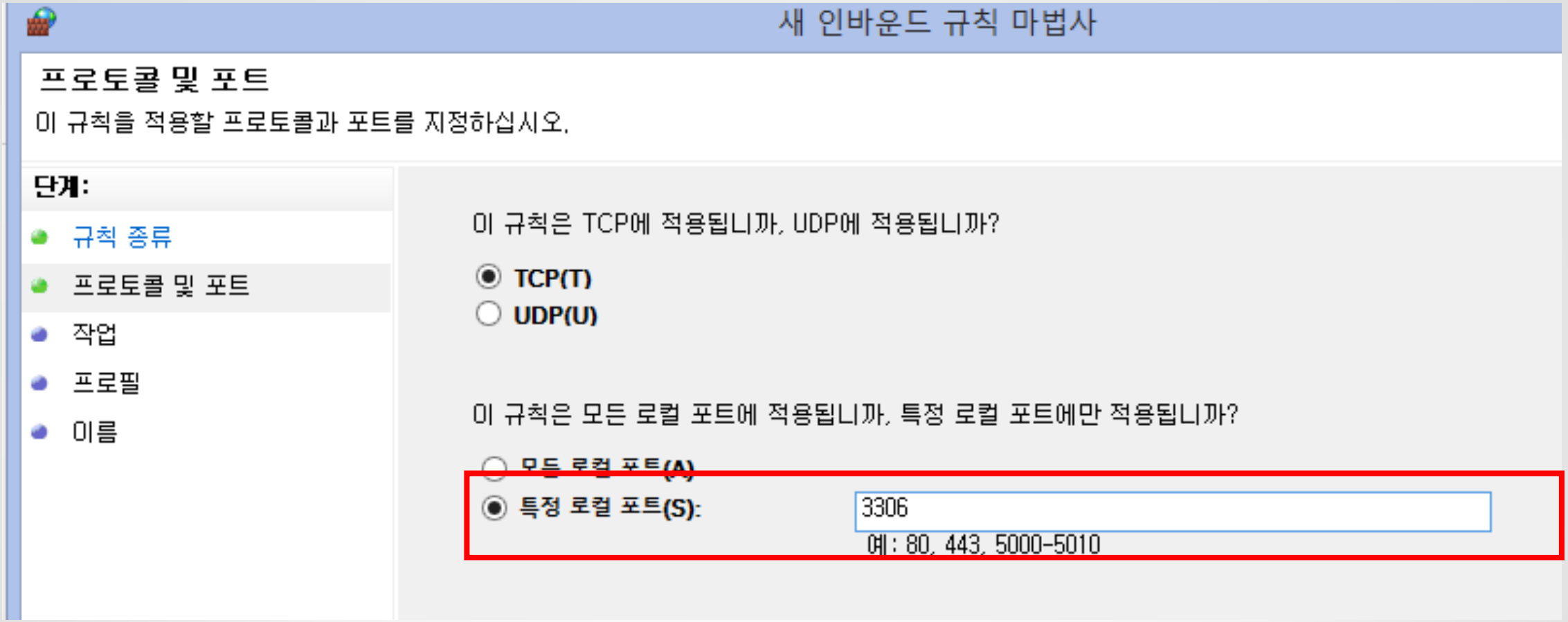
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙>>새규칙>>포트**



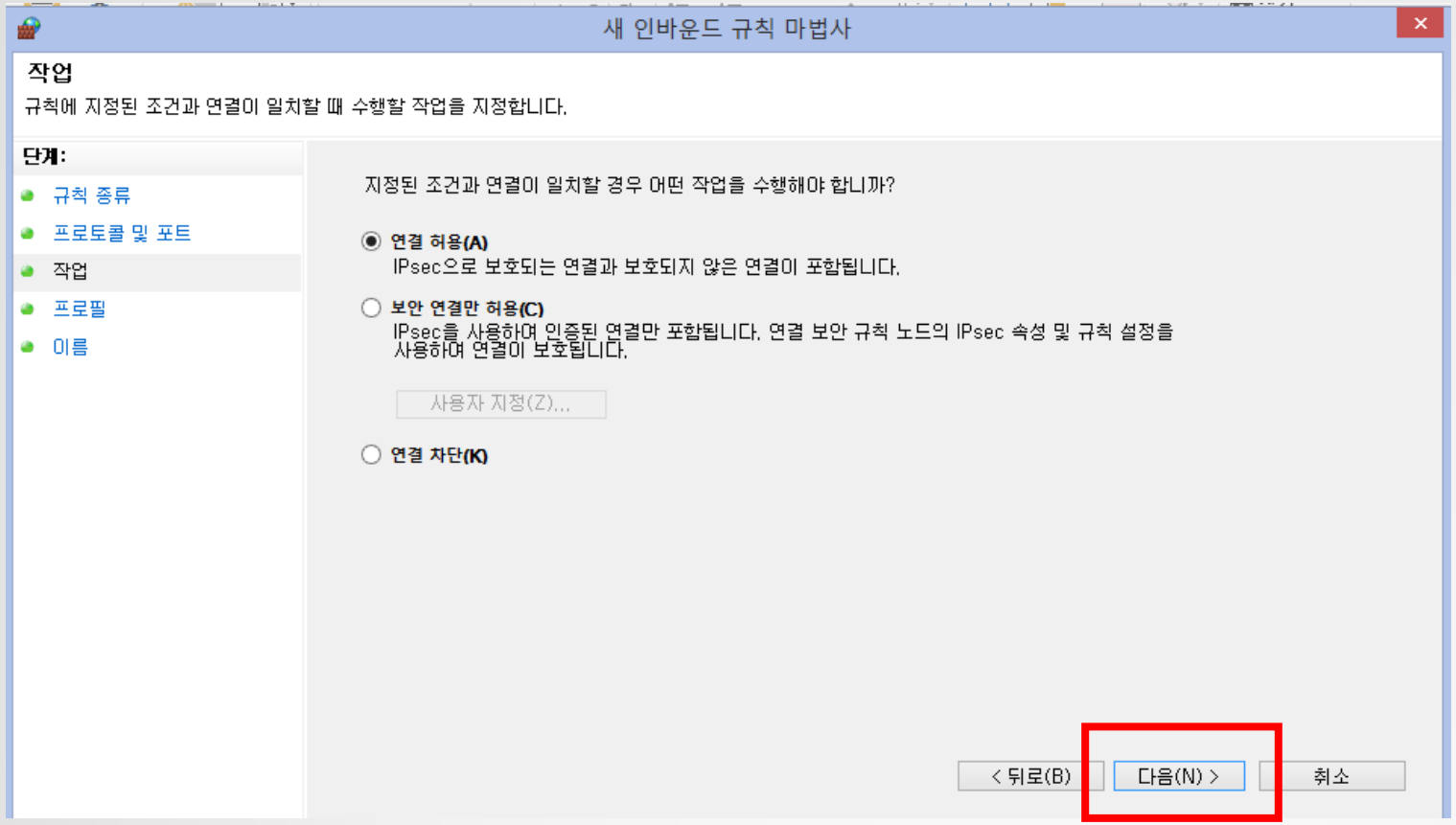
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙>>새규칙>>포트>>**
**특정로컬포트 3306**



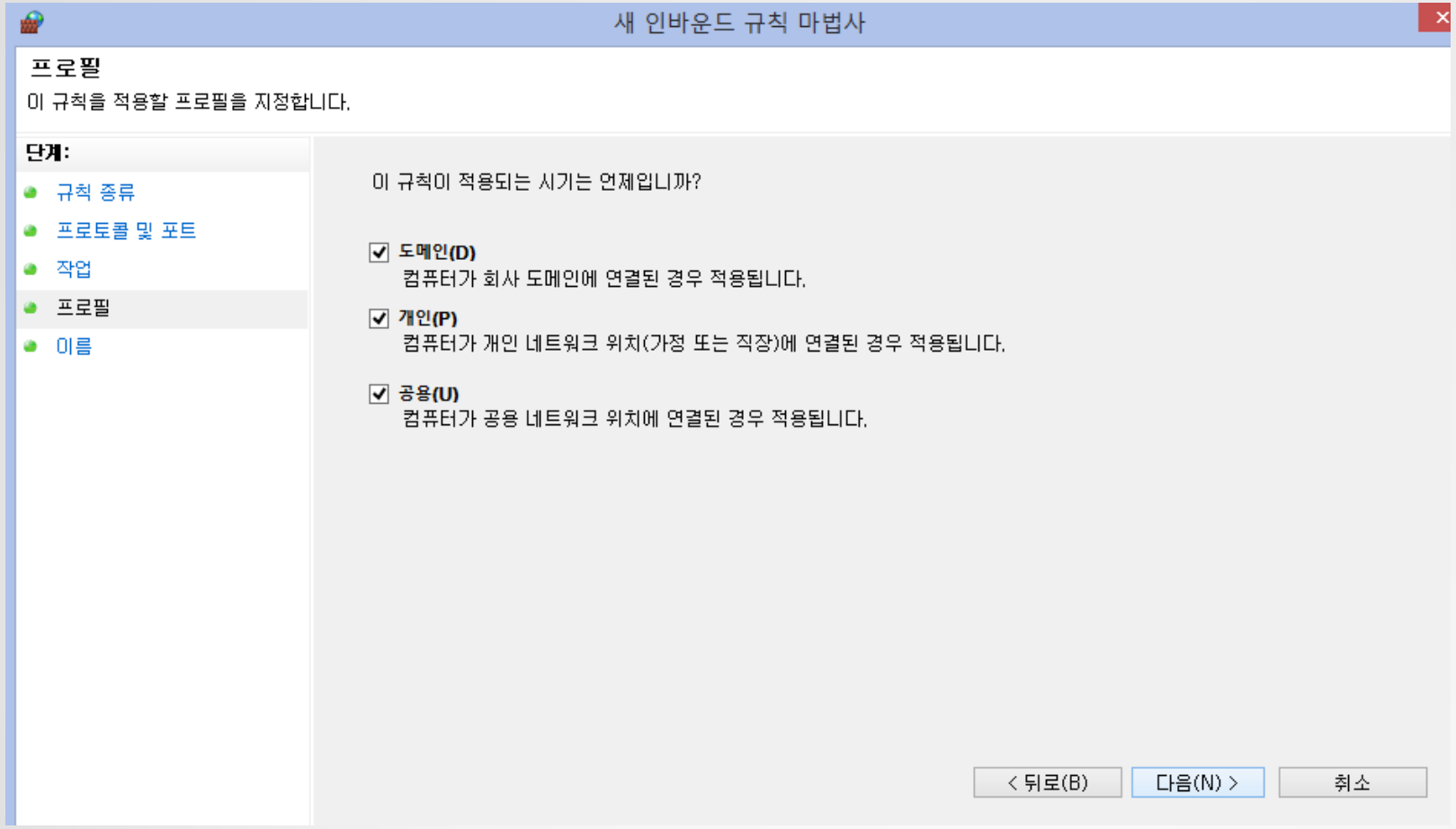
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙>>새규칙>>포트>>**
**특정로컬포트 3306**



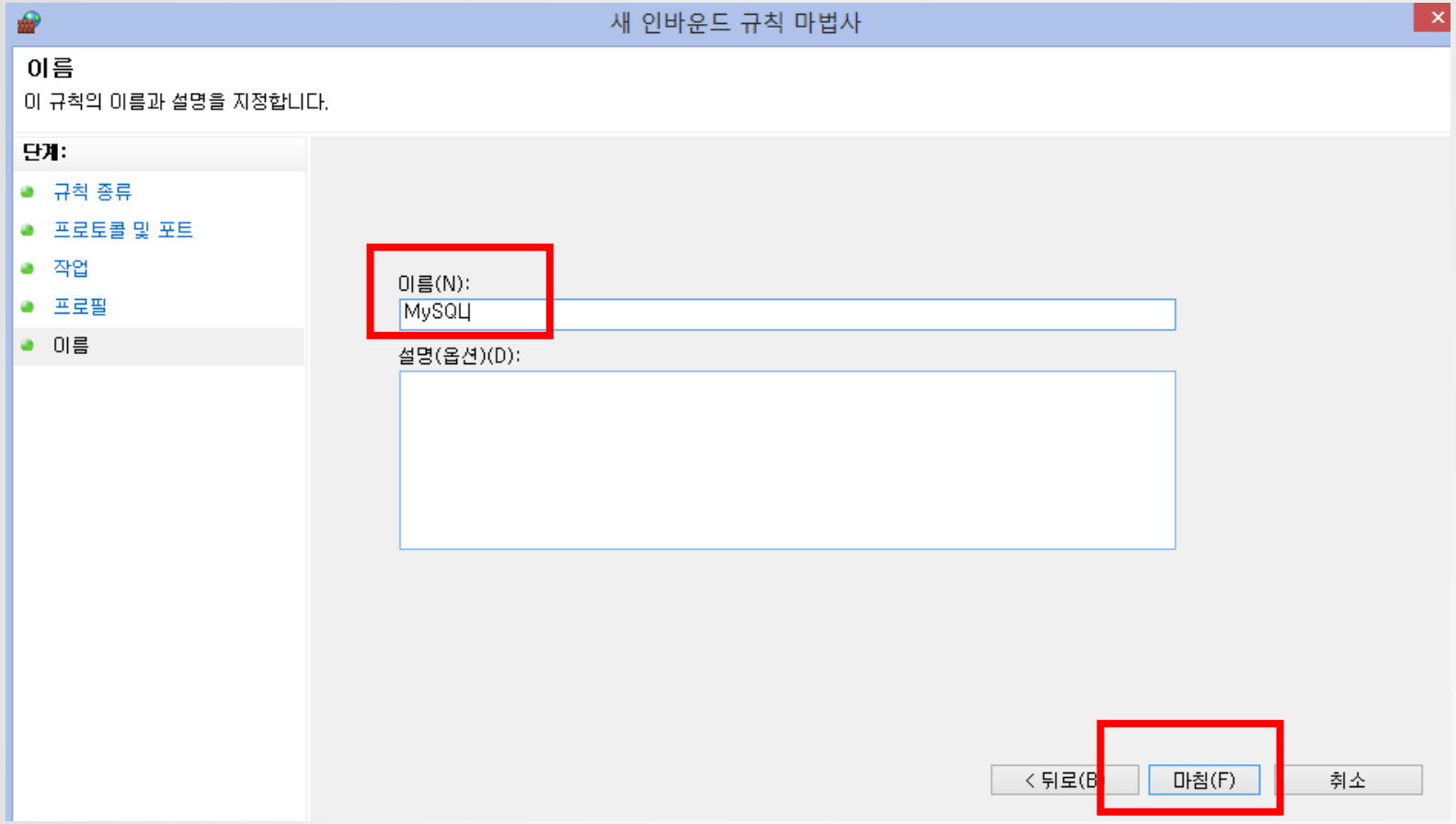
# MySQL의 원격 접속

**접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.**
**제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙>>새규칙>>포트>>**
**특정로컬포트 3306**



# MySQL의 원격 접속

접속허가 했으나 원격접속이 안될때 windows방화벽을 체크해 보자.
제어판>>시스템 및 보안>>Windows 방화벽>> 고급설정>>인바운드 규칙>>새규칙>>포트>>
특정로컬포트 3306>>이름 MySQL >>마침



## Mission

1. A씨가 CREATE TAVLE 로 생성한 빈 테이블 Goods에 대해 다음 SQL문을 실행하여 데이터를 등록했다.

```
-- A씨가 실행한 SQL (SQL Server, PostgreSQL)
BEGIN TRANSACTION;
   INSERT INTO Goods VALUES ('0001', '티셔츠', '의류', 1000, 500, '2009-09-20');
   INSERT INTO Goods VALUES ('0002', '펀칭기', '사무용품', 500, 320, '2009-09-11');
   INSERT INTO Goods VALUES ('0003', '와이셔츠', '의류', 4000, 2800, NULL);

-- A씨가 실행한 SQL (MySQL)
START TRANSACTION;
   INSERT INTO Goods VALUES ('0001', '티셔츠', '의류', 1000, 500, '2009-09-20');
   INSERT INTO Goods VALUES ('0002', '펀칭기', '사무용품', 500, 320, '2009-09-11');
   INSERT INTO Goods VALUES ('0003', '와이셔츠', '의류', 4000, 2800, NULL);

-- A씨가 실행한 SQL (Oracle, DB2)
   INSERT INTO Goods VALUES ('0001', '티셔츠', '의류', 1000, 500, '2009-09-20');
   INSERT INTO Goods VALUES ('0002', '펀칭기', '사무용품', 500, 320, '2009-09-11');
   INSERT INTO Goods VALUES ('0003', '와이셔츠', '의류', 4000, 2800, NULL);
```

바로이어 B씨가 다른 컴퓨터로부터 동일한 데이터베이스 접속해서 다음 SQL문을 실행했다.
이때 B씨는 어떤 결과를 볼 수 있는가?

```
SELECT * FROM Goods;
```

# Mission3

2. 다음 3행이 포함된 Goods테이블이 있다.

| goods_id | goods_name | goods_classify | sell_price | buy_price | register_date |
|---|---|---|---|---|---|
| 0001 | 티셔츠 | 의류 | 1000 | 500 | 2010-01-01 |
| 0002 | 펀칭기 | 사무용품 | 500 | 320 | 2010-01-11 |
| 0003 | 와이셔츠 | 의류 | 4000 | 2800 | |

이 3행을 그대로 복사해서 6행으로 늘리려고 생각하고 다음 INSERT 문을 실행했다.
결과는 어떻게 되는가?

```
INSERT INTO Goods SELECT * FROM Goods;
```

3. 위의 2번에서 사용한 Goods테이블이 있다. 이와 별도로 새로운 테이블을 하나더 만드는데 ,
다음과 같이 차익열을 가진 GoodsProfit 테이블을 만든다. 이테이블에 Goods테이블로부터 차익을 계산해서
다음과 같은 데이터를 등록하는 SQL문을 작성.

```
CREATE TABLE GoodsProfit
(goods_id    CHAR(4)       NOT NULL,
 goods_name  VARCHAR(100)  NOT NULL,
 sell_price INTEGER,
 buy_price INTEGER,
 profit        INTEGER,
 PRIMARY KEY(goods_id));
```

| goods_id | goods_name | sell_price | buy_price | profit |
|---|---|---|---|---|
| 001 | 티셔츠 | 1000 | 500 | 500 |
| 002 | 펀칭기 | 500 | 320 | 180 |
| 003 | 와이셔츠 | 4000 | 2800 | 1200 |

## 참고문헌


- SQL 더쉽게 , 더깊게 -Jpub

# Thank you


ORACLE
firebird
PostgreSQL
IBM DB2
Microsoft SQL Server 2008
SQLite
MySQL
123
TERADATA
CSV